



Security Review For Enhanced



Collaborative Audit Prepared For:
Lead Security Expert(s):

Enhanced
1337web3
PeterSR

Date Audited:

vinica_boy
May 26 - June 17, 2026

Introduction

Enhanced is a structured-product infrastructure for onchain assets, including real-world assets (RWAs). In v1, the system allows users to write options such as covered calls and cash-secured puts onchain, as well as a vault infrastructure that enables the rolling of these strategies.

The core consists of 2 main components: an options engine allowing request-for-quote (RFQ) where users can write options onchain directly, and automated vaults, which pool depositor capital and execute through the same engine. Both resolve to identical minting, collateral, and settlement logic.

The protocol builds on a battle-tested onchain options protocol, adding a signed RFQ auction, the rolling vault layer, and improved modular access controls.

For the vault, the design innovates to align with the user's favor. In a first for options vault products, premiums can accrue to a separately withdrawable balance – income can be taken every epoch without touching principal. Parameters such as epoch cadence and fees are fully configurable by the vault owner. Options are written via auction to competing institutional market makers rather than priced by a single desk – allowing better price discovery, resulting in better premiums reaching the user.

Enhanced v1 is an onchain system that opens up more possibilities for structured payouts and strategies to come.

Scope

Repository: Enhanced-Finance/contracts

Audited Commit: 4e55bcfa33a0ebca729f81cb4f2af4eaaf548365

Final Commit: eb2167e14bfc9794ea392dadf39f7fe75a7f0777

Files:

- src/core/AddressBook.sol
- src/core/ControllerLogic.sol
- src/core/Controller.sol
- src/core/EnhancedOptions.sol
- src/core/interfaces/AddressBookInterface.sol
- src/core/interfaces/ControllerInterface.sol
- src/core/interfaces/ControllerLogicInterface.sol
- src/core/interfaces/EnhancedPricerInterface.sol
- src/core/interfaces/IAavePool.sol
- src/core/interfaces/IController.sol

- src/core/interfaces/IEnhancedOptions.sol
- src/core/interfaces/IOtokenFactory.sol
- src/core/interfaces/IOtoken.sol
- src/core/interfaces/ISwapRouter.sol
- src/core/interfaces/MarginCalculatorInterface.sol
- src/core/interfaces/MarginPoolInterface.sol
- src/core/interfaces/OracleInterface.sol
- src/core/interfaces/OtokenInterface.sol
- src/core/interfaces/WhitelistInterface.sol
- src/core/libs/Actions.sol
- src/core/libs/FixedPointInt256.sol
- src/core/libs/MarginVault.sol
- src/core/libs/MMarketOperations.sol
- src/core/libs/Parser.sol
- src/core/ManualPricer.sol
- src/core/MarginCalculator.sol
- src/core/MarginPool.sol
- src/core/MMarket.sol
- src/core/Oracle.sol
- src/core/OtokenFactory.sol
- src/core/Otoken.sol
- src/core/packages/BokkyPooBahsDateTimeLibrary.sol
- src/core/Whitelist.sol
- src/periphery/vault/EnhancedVault.sol
- src/periphery/vault/interfaces/IEnhancedVault.sol
- src/periphery/vault/libs/EnhancedVaultCycleLib.sol
- src/periphery/vault/libs/EnhancedVaultRecordsLib.sol

Final Commit Hash

eb2167e14bfc9794ea392dadf39f7fe75a7f0777

Findings

Each issue has an assigned severity:

- High issues are directly exploitable security vulnerabilities that need to be fixed.
- Medium issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.
- Low/Info issues are non-exploitable, informational findings that do not pose a security risk or impact the system's integrity. These issues are typically cosmetic or related to compliance requirements, and are not considered a priority for remediation.

Issues Found

High	Medium	Low/Info
1	7	26

Issues Not Fixed and Not Acknowledged

High	Medium	Low/Info
0	0	0

Issue H-1: Physical-settlement payout depends on writer settlement order when multiple writers share an oToken [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-05-enhanced-may-26th-2026/issues/80>

Summary

Physically settled positions share per-oToken redemption balances (collateral, receiving asset, oToken quantity), and each writer's settlement is computed against whatever remains in those balances at the moment they settle. Because the oToken is keyed only by option-series parameters, multiple writers of the same series share one oToken and one pooled set of balances. When buyers only partially exercise, the remaining balances are a mix of collateral and strike asset, and the order in which writers settle determines how much of each asset they receive. Writers with identical positions can receive materially different value.

Vulnerability Detail

The oToken address is resolved purely from the option series:

```
address otokenFromFactory = factory.getOtoken(  
    otoken.underlying,  
    otoken.strikeAsset,  
    otoken.collateral,  
    otoken.strike,  
    otoken.expiration,  
    otoken.isPut,  
    otoken.isPhysicallySettled  
);
```

There is no vault-owner component, so any writer minting the same series writes the same oToken and shares its redemption balances.

handleSettle then distributes the shared balances per writer based on the live balances. The strike asset is paid out first-come, and the collateral share is computed against the current (shrinking) totals:

```
if (settleMem.receivingAssetBalance == 0) {  
    // scenario 1: no strike asset left, pay entirely in collateral  
    uint256 vaultShareOfCollateralRedemptionBalance =  
        settleMem.collateralRedemptionBalance * settleMem.shortAmount /  
        ↪ settleMem.otokenQuantity;  
    ...  
}
```

```

} else if (settleMem.strikePayout > settleMem.receivingAssetBalance) {
    // scenario 2: not enough strike asset, take what is left + collateral share
    settleMem.strikePayout = settleMem.receivingAssetBalance;
    ...
    uint256 vaultShareOfCollateralRedemptionBalance =
        ↪ settleMem.collateralRedemptionBalance * contractsLeft
        / (settleMem.otokenQuantity - settleMem.strikeCount);
    ...
}

```

The first writer to settle absorbs the available strike asset; later writers settle against the depleted strike balance and receive proportionally more collateral. When the strike asset and the collateral have diverged in value (the normal ITM case), this is not value-neutral.

Worked example (CALL, underlying = collateral = WETH, strike = 1000, expiry price = 2000, two writers each minted 5 oTokens):

Initial shared balances: collateral = 10 WETH, receivingAsset = 0, otokenQty = 10e8. Four buyers exercise during the redeem window, each paying 1000 USDC into the pool and taking 1 WETH:

```
collateral = 6 WETH, receivingAsset = 4000 USDC, otokenQty = 10e8
```

First settlement (writer A, 5 oTokens): $\text{strikePayout} = 1000 * 5 = 5000 > 4000$ (scenario 2). Takes all 4000 USDC ($\text{strikeCount} = 4$), collateral share = $6 * (5-4) / (10-4) = 1$ WETH. Writer A receives 4000 USDC + 1 WETH = \$6,000. Remaining: collateral = 5 WETH, receivingAsset = 0, otokenQty = 5e8.

Second settlement (writer B, 5 oTokens): receivingAsset = 0 (scenario 1). Collateral share = $5 * 5 / 5 = 5$ WETH. Writer B receives 5 WETH = \$10,000.

Both writers held identical positions, but A receives \$6,000 and B receives \$10,000 purely because of settlement order. The fair pro-rata split of the 6 WETH + 4000 USDC remaining (= \$16,000) would be \$8,000 each.

Impact

When multiple writers share an oToken series and buyers exercise only partially, settlement is not pro-rata: the writer(s) who settle first are forced to take the less valuable mix of assets, and later writers receive the more valuable mix. This is a value transfer between writers driven by transaction ordering, not by their actual positions, and can amount to a significant fraction of a writer's payout (about 25% in the example). It is a fairness/loss issue between users rather than a drain of protocol funds, and it requires multiple writers of the same series plus partial exercise.

Code Snippet

<https://github.com/sherlock-audit/2026-05-enhanced-may-26th-2026/blob/b2e0082f695d5f0a82d8a5051d69423ad4a7cba4/contracts/src/core/EnhancedOptions.sol#L835-L860>

<https://github.com/sherlock-audit/2026-05-enhanced-may-26th-2026/blob/b2e0082f695d5f0a82d8a5051d69423ad4a7cba4/contracts/src/core/ControllerLogic.sol#L482-L530>

Tool Used

Manual Review

Recommendation

Make the redemption balances independent per writer so one writer's settlement cannot consume another writer's share, or compute each writer's payout strictly pro-rata against the original totals rather than the live (depleted) balances.

Discussion

6xiaowu9

Fixed in commit 4c33236 (Isolate oTokens by vault owner).

To address this issue, I added vaultOwner to the oToken definition and included it in the oToken factory's unique key. As a result, otherwise identical option series written by different vault owners now use different oTokens and therefore maintain independent physical-settlement redemption balances. This prevents one writer's settlement from consuming balances that belong to another writer and removes cross-writer settlement-order dependence. I intentionally did not include vaultId in the oToken key. The business logic aggregates positions from multiple vaults belonging to the same owner for the same option series. Adding vaultId would unnecessarily split those positions into separate oTokens and prevent this aggregation. Therefore, oToken uniqueness is now based on the option-series parameters plus vaultOwner, while multiple vaults owned by the same account can continue sharing and merging the same position.

btw, I think this should only be considered a design-level fairness issue. Also, when settling, the operator will settle the vaults in the corresponding order, which is fair as well.

Therefore, I do not think this should be classified as High.

Issue M-1: Buyback does not update totalDeposited or initialAmountTotal, drifting vault accounting [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-05-enhanced-may-26th-2026/issues/41>

Summary

The buyback function allocates collateral to users via `_allocateBuybackToUsers` which increments `fund.pendingActivePrincipal` without updating `st.totalDeposited` or `fund.initialAmountTotal`. This causes two cascading failures: a capacity bypass and a circuit-breaker bypass.

Vulnerability Detail

When buyback runs, it swaps user premium for collateral and distributes shares to users:

```
// EnhancedVault.sol:873
fund.pendingActivePrincipal += shares[i];
```

Neither `st.totalDeposited` nor `fund.initialAmountTotal` is incremented here.

Capacity bypass: `totalDeposited` is the running total used to enforce the vault capacity ceiling:

```
// EnhancedVault.sol:527
if (st.totalDeposited + amount > st.params.capacity) revert CapacityExceeded();
```

Because buyback does not increment `totalDeposited`, the vault can hold more collateral than capacity after a buyback. New depositors can enter up to the original capacity ceiling even though actual holdings exceed it.

Circuit-breaker bypass: `_isBelowMinPrincipalRatio` uses `fund.initialAmountTotal` as the denominator:

```
// EnhancedVault.sol:1330
uint256 principalRatio = principal * RATIO_BASE / initialAmountTotal;
```

Because buyback does not increase `initialAmountTotal`, a user who received buyback compensation has their ratio computed against a stale (too-small) denominator, making them appear above the `minPrincipalRatio` threshold even when their true loss ratio warrants automatic pausing. The `systemPauseFunds` protection is silently bypassed.

Impact

- Vault capacity can be silently exceeded after buyback, breaking the capacity invariant.
- Users who should be ejected by the circuit-breaker are not, because their health ratio is overstated.

Code Snippet

```
// EnhancedVault.sol _allocateBuybackToUsers
fund.pendingActivePrincipal += shares[i]; // totalDeposited and initialAmountTotal
↪ NOT updated
```

Tool Used

Manual Review

Recommendation

In `_allocateBuybackToUsers`, for each user increment both `fund.initialAmountTotal += shares[i]` and `st.totalDeposited += shares[i]` to keep the accounting consistent with a regular deposit.

Discussion

6xiaowu9

Fixed. commit: 6b32cbf.fix: buyback amount should count toward the cap.

The initial principal amount should not be modified here.

Issue M-2: systemPauseFunds uses settledActive numerator while queue processing uses totalCandidate, creating inconsistent circuit-breaker enforcement [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-05-enhanced-may-26th-2026/issues/46>

Summary

systemPauseFunds evaluates the minimum principal ratio using only settledActive as the numerator, while _processSingleQueuedUser uses totalCandidate = settledActive + pendingActivePrincipal + systemPausedPrincipal. A user who received buyback compensation that brings their effective balance above the threshold will be incorrectly ejected by systemPauseFunds but correctly retained by normal queue processing.

Vulnerability Detail

systemPauseFunds:

```
// EnhancedVault.sol:1268-1269
uint256 settledActive = _projectSettledActive(fund, cycleCumCollateral);
if (!_isBelowMinPrincipalRatio(st, settledActive, fund.initialAmountTotal)) {
    revert UserNotBelowMinPrincipalRatio(user);
}
```

_processSingleQueuedUser:

```
// EnhancedVault.sol:959-971
uint256 totalCandidate = candidateActive + fund.systemPausedPrincipal;
} else if (_isBelowMinPrincipalRatio(st, totalCandidate, fund.initialAmountTotal)) {
```

totalCandidate includes pendingActivePrincipal (which is where buyback allocations land). settledActive does not. A user who received buyback compensation sufficient to bring their ratio above the threshold will pass the queue processing check but fail the systemPauseFunds check – their fate depends entirely on which the operator calls first.

Impact

Users who benefited from the vault's own buyback mechanism can be incorrectly ejected by systemPauseFunds, penalizing them for receiving compensation the vault itself provided. The two functions enforce fundamentally different circuit-breaker thresholds.

Code Snippet

```
// systemPauseFunds uses settledActive only
if (!_isBelowMinPrincipalRatio(st, settledActive, fund.initialAmountTotal))

// queue processing uses totalCandidate (includes buyback)
} else if (_isBelowMinPrincipalRatio(st, totalCandidate, fund.initialAmountTotal))
```

Tool Used

Manual Review

Recommendation

Align systemPauseFunds to use the same numerator as queue processing: totalCandidate = settledActive + fund.pendingActivePrincipal + fund.systemPausedPrincipal.

Discussion

6xiaowu9

Fixed. Commit: 3c3e1fe fix: account pending principal in system pause

Issue M-3: Ghost `initialAmountTotal` permanently blocks re-entry after losses [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-05-enhanced-may-26th-2026/issues/47>

Summary

`initialAmountTotal` accumulates every deposit but is only decremented by actual withdrawals – losses are never subtracted. After a user suffers losses and exits, a residue equal to total losses persists in `initialAmountTotal`. Any future re-deposit is evaluated against this inflated denominator, making re-entry require a fresh deposit equal in size to all prior losses.

Vulnerability Detail

`initialAmountTotal` is incremented on deposit and decremented only when funds are physically withdrawn:

```
// EnhancedVault.sol:537
fund.initialAmountTotal += amount; // on deposit

// EnhancedVault.sol:564, 595, 721
fund.initialAmountTotal -= rec.amount; // on withdrawal
```

When a user suffers losses, those losses are never subtracted from `initialAmountTotal`. After a full exit, the ghost residue remains.

Concrete example (as described in `bugs.md`):

- Alice deposits 1000 USDC. `initialAmountTotal` = 1000.
- 60% loss. Alice's principal = 400. Ratio = 40% < 50% threshold → ejected.
- Alice withdraws 400 USDC. `initialAmountTotal` = 1000 - 400 = 600 (loss residue).
- Alice tries to re-enter with 100 USDC. `initialAmountTotal` = 700.
- Ratio = 100/700 = 14.3% < 50% → ejected immediately.
- Required re-entry deposit to reach 50%: $Y / (600 + Y) \geq 0.5 \rightarrow Y \geq 600$ **USDC**.

Alice must deposit an amount equal to her entire historical losses just to re-enter.

Impact

Users who suffered losses and exited are permanently penalized on re-entry. The required deposit scales with historical losses, not current vault risk. This is not disclosed

to users and creates an undocumented barrier to participation.

Code Snippet

```
// Loss is never deducted from initialAmountTotal
// After full exit, ghost residue = total losses
fund.initialAmountTotal += amount; // grows on deposit
// no subtraction for losses
```

Tool Used

Manual Review

Recommendation

When a user's position is settled at a loss ($\text{collateralRatio} < 1$), subtract the loss amount from `initialAmountTotal`: `fund.initialAmountTotal -= (activePrincipal - settledActive)`. This ensures the denominator reflects actual invested capital, not historical deposits.

Discussion

6xiaowu9

This isn't a problem. If `initialAmountTotal` is modified after a loss, it will no longer be the initial principal, causing the `minPrincipalRatio` function to malfunction.

PeterSRWeb3

Thank you for the clarification. I agree that `initialAmountTotal` is intended as a reference point for loss-awareness, and I understand the reasoning – if you reset it after losses, the `minPrincipalRatio` check loses its meaning while a position is still active.

However, the issue is specifically in the full-exit-and-re-entry path, not in the mid-position case. After a user fully withdraws (zero active principal, zero pending, all funds out), `initialAmountTotal` retains the loss residue permanently because `fund.exists` is never reset and there is no zeroing path. At that point the user has no position in the vault, so the loss-tracking purpose of `initialAmountTotal` has already been served – the ejection already happened. The ghost denominator then carries forward into a completely new deposit as if it were part of the new position's risk basis, which it is not.

The concrete result is that re-entry cost scales with historical losses rather than new position size. A user who lost 10,000 USDC, fully exited, and wants to open a fresh small position must deposit 10,000 USDC just to pass the threshold – not because their new position warrants it, but because the accounting from a prior closed position was never

cleared. I believe this specific path – full exit followed by re-entry – is where the design intent and the implementation diverge, and that is the narrow claim we are making.

6xiaowu9

Thank you for the clarification. We agree that the full-exit-and-re-entry path could retain a stale loss basis and incorrectly apply it to a new position. We addressed this by reducing `initialAmountTotal` proportionally whenever principal exits: $\text{basisReduction} = \text{initialAmountTotal} \times \text{exitAmount} / \text{principalBeforeExit}$ This preserves the loss ratio during partial withdrawals. When the user exits all remaining principal, `initialAmountTotal` is explicitly cleared, preventing historical losses from affecting future deposits.

Regression tests cover: Partial withdrawal after a loss preserves the principal ratio. Full withdrawal after a loss clears `initialAmountTotal`. Full exit through system pause clears the basis. `claimActive` after a loss clears the basis. Re-entry therefore starts with a fresh principal basis. Fixed in commit e79630e.

Issue M-4: Fee is not included in EIP-712 type strings — operator can drain maker's MMarket balance [ACKNOWLEDGED]

Source:

<https://github.com/sherlock-audit/2026-05-enhanced-may-26th-2026/issues/50>

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

The `fee` parameter is extracted from the transaction payload and deducted from the maker's MMarket balance on every trade, but it is absent from both EIP-712 type strings. The maker's signature never commits to any specific fee value. The operator can set `fee` to the maker's entire MMarket balance in any payload submission, draining it to `feeRecipient` without the maker's knowledge or consent.

Vulnerability Detail

The fee is extracted from the raw payload:

```
// Parser.sol:142
fee := mload(add(payload, 361))
```

But neither EIP-712 type string includes `fee`:

```
// EnhancedOptions.sol:149
"Quote(address assetAddress,uint256 chainId,bool isPut,bool
↪ isPhysicallySettled,uint256 strike,uint64 expiry,address maker,uint64
↪ nonce,uint256 price,uint256 quantity,bool isTakerBuy,uint64 validUntil,address
↪ usd,address collateralAsset)"

// EnhancedOptions.sol:152
"Confirmation(address maker,address assetAddress,uint256 chainId,uint64 expiry,bool
↪ isPut,bool isPhysicallySettled,uint64 nonce,uint256 price,uint256
↪ quantity,uint64 quoteNonce,bytes quoteSignature,uint256 strike,address
↪ taker,bool isTakerBuy,address usd,address collateralAsset,uint256
↪ collateralAmount)"
```

The fee is then withdrawn from the maker's MMarket balance:

```
// EnhancedOptions.sol:685-695
if (fee > 0) {
    operations[3] = MMarketOperations.Operation(
        MMarketOperations.OperationType.Withdraw,
        sellerConfirmation.maker,
```

```

        feeRecipient,
        sellerConfirmation.usd,
        ZERO_ADDRESS,
        fee,
        0,
        bytes("")
    );
}

```

Since `fee` is not signed, the operator can inject any value into the payload bytes at offset 361 and the maker's signature remains valid.

Impact

The operator can drain the maker's entire MMarket USDC balance to `feeRecipient` on any trade submission, without the maker's knowledge or consent. This is a direct theft vector requiring only operator access.

Code Snippet

```

// fee extracted from payload but never signed
fee := mload(add(payload, 361))
// maker signature does not cover fee - no binding commitment

```

Tool Used

Manual Review

Recommendation

Add `uint256 fee` to both the `Quote` and `Confirmation` EIP-712 type strings and include it in the digest computation, so the maker's signature commits to the exact fee amount.

Discussion

6xiaowu9

This is not an issue; it is intended by design.

Currently, we only charge fees to the maker. However, our overall order flow is asynchronous. There may be time gaps between the steps, including RFQ, quotation, taker confirmation and signature, maker confirmation and signature, and order submission on-chain.

There may also be timing differences between these steps and the fee policy changes we are going to announce, which could potentially lead to certain issues.

We will also enter into agreements with makers to confirm and acknowledge this matter.

PeterSRWeb3

Yes, this issues can be ack.

cwkang1998

To add more clarity:

As maker fees are subject to change based on the market condition and the available liquidity and volume of the protocol, given the asynchronous nature of the the RFQ flow, we require this flexibility to adjust fees according to changes in agreements with the makers.

These operation are restricted to operator, which is a trusted privileged role in the protocol. Additionally the operation submitter would not be able to easily set a new `feeRecipient` without also gaining the owner privileged role access, which is separated from the operator role.

As such we acknowledge this issue, however no code changes is planned as this is an intentional requirement of the product. If in the future we ever remove the trust assumption on the operator, this shall be revised accordingly.

Issue M-5: validUntil Field Is Not Verified Allowing Stale Quote Execution [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-05-enhanced-may-26th-2026/issues/57>

Summary

Quote expiration times (`validUntil`) are cryptographically committed to in EIP-712 signatures but never validated on-chain, allowing expired quotes to be executed indefinitely after the maker intended them to expire.

Vulnerability Detail

The `validUntil` field is included as part of the EIP-712 digest that the maker signs, establishing a cryptographic commitment to the quote's validity period:

```
string internal constant QUOTE_TYPE =
    "Quote(address assetAddress,uint256 chainId,bool isPut,bool
    ↪ isPhysicallySettled,uint256 strike,uint64 expiry,address maker,uint64
    ↪ nonce,uint256 price,uint256 quantity,bool isTakerBuy,uint64
    ↪ validUntil,address usd,address collateralAsset)";
```

However, in the `_doNewUserPosition` function and related entry points (`ingressoNewTrustedTakerPosition`, `_doNewTrustedMakerPosition`, `ingressoNewTrustedTakerAndMakerPosition`), the signature is verified but the `validUntil` timestamp is never checked:

```
function _doNewUserPosition(bytes calldata payload) internal {
    // ... parsing code ...

    bytes32 quoteDigest = getQuoteDigest(mmQuote);
    bytes32 confDigest = getConfirmationDigest(sellerConfirmation);

    _consumeDigest(quoteDigest);
    _consumeDigest(confDigest);
    // @audit-find: High, Missing require(block.timestamp <= mmQuote.validUntil,
    ↪ QuoteExpired())
    // validUntil is committed to in the EIP-712 digest but never checked on-chain

    if (!SignatureChecker.isValidSignatureNow(mmQuote.maker, quoteDigest,
    ↪ quoteSig)) {
        revert InvalidQuoteSignature();
    }
    // ... execution continues without validating validUntil ...
}
```

The maker intends the quote to be valid only until `validUntil`, but this constraint is not enforced, creating a mismatch between the maker's intent and the contract's behavior.

Impact

1. **Stale Quote Execution:** Quotes can be executed indefinitely after the maker intended them to expire, potentially days, weeks, or months later.
2. **Adverse Market Conditions:** A quote signed at one time with prices valid for a specific window can be executed when market conditions have changed dramatically, locking the maker into outdated pricing.
3. **Economic Exploitation:** Malicious or opportunistic actors can monitor pending quotes and execute them at strategic times when market conditions favor execution over the maker's original intent.
4. **Maker Loss of Control:** While makers sign with the expectation their quote expires, they have no actual expiration protection, undermining the security model of time-sensitive trading.
5. **Systemic Risk:** In volatile markets, old quotes executed at stale prices could cause cascading liquidations or vault instability.

Code Snippet

The vulnerability exists in multiple entry point functions in `EnhancedOptions.sol`:

- `_doNewUserPosition`
- `ingressoNewTrustedTakerPosition`
- `_doNewTrustedMakerPosition`
- `ingressoNewTrustedTakerAndMakerPosition`

All four functions parse and verify the quote signature but omit the expiration check.

Tool Used

Manual Review

Recommendation

Add an explicit on-chain validation of the quote expiration time in all four vulnerable functions. The check should occur after signature verification but before execution:

```
function _doNewUserPosition(bytes calldata payload) internal {
    (
        Parser.Quote memory mmQuote,
```

```

        Parser.Confirmation memory sellerConfirmation,
        bytes memory quoteSig,
        bytes memory confSig,
        uint256 fee
    ) = Parser.parseQuoteAndConfirmation(payload);

    bytes32 quoteDigest = getQuoteDigest(mmQuote);
    bytes32 confDigest = getConfirmationDigest(sellerConfirmation);

    _consumeDigest(quoteDigest);
    _consumeDigest(confDigest);

    // ADD THIS CHECK
    require(block.timestamp <= mmQuote.validUntil, "QuoteExpired");

    if (!SignatureChecker.isValidSignatureNow(mmQuote.maker, quoteDigest,
        ↪ quoteSig)) {
        revert InvalidQuoteSignature();
    }
    // ... rest of function
}

```

Apply the same `require(block.timestamp <= mmQuote.validUntil)` check to:

- `ingressoNewTrustedTakerPosition`
- `_doNewTrustedMakerPosition`
- `ingressoNewTrustedTakerAndMakerPosition`

Alternatively, extract this into a helper function to avoid duplication:

```

function _validateQuoteExpiration(Parser.Quote memory mmQuote) internal view {
    require(block.timestamp <= mmQuote.validUntil, "QuoteExpired");
}

```

Then call `_validateQuoteExpiration(mmQuote)` in all four functions after digest consumption but before signature verification.

Discussion

6xiaowu9

I believe this issue is invalid under the intended execution model.

`validUntil` is part of the signed quote payload and binds the maker's quote context, but the maker quote alone is not an executable order authorization.

A position can only be created after the full multi-party authorization flow is completed:

1. the taker requests a quote;

2. the maker responds with a signed quote;
3. the taker confirms the actual execution quantity and signs the confirmation;
4. the maker-side payer approves the payment, signs the payment authorization, and grants the required authorization;
5. the completed order package is submitted to the platform backend/operator;
6. the backend/operator performs final validation and submits the on-chain transaction.

Therefore, an old maker quote cannot be executed by itself. The report assumes a permissionless stale-quote execution path, but such a path does not exist. Execution requires the later taker confirmation, maker-side payer approval/signature/authorization, nonce consumption, and operator-side validation.

For this reason, `validUntil` is not enforced as an unconditional on-chain expiry check. Stale-quote handling is part of the platform/operator validation policy over the complete order package, not a standalone on-chain check against the maker quote timestamp.

Adding `require(block.timestamp <= quote.validUntil)` would be a product-policy change to the execution workflow, not a vulnerability fix.

zyppyvids

I believe this issue is invalid under the intended execution model.

`validUntil` is part of the signed quote payload and binds the maker's quote context, but the maker quote alone is not an executable order authorization.

A position can only be created after the full multi-party authorization flow is completed:

1. the taker requests a quote;
2. the maker responds with a signed quote;
3. the taker confirms the actual execution quantity and signs the confirmation;
4. the maker-side payer approves the payment, signs the payment authorization, and grants the required authorization;
5. the completed order package is submitted to the platform backend/operator;
6. the backend/operator performs final validation and submits the on-chain transaction.

Therefore, an old maker quote cannot be executed by itself. The report assumes a permissionless stale-quote execution path, but such a path does not exist. Execution requires the later taker confirmation, maker-side payer approval/signature/authorization, nonce consumption, and operator-side validation.

For this reason, `validUntil` is not enforced as an unconditional on-chain expiry check. Stale-quote handling is part of the platform/operator validation policy over the complete order package, not a standalone on-chain check against the maker quote timestamp.

Adding `require(block.timestamp <= quote.validUntil)` would be a product-policy change to the execution workflow, not a vulnerability fix.

Thank you for the answer! I agree that currently in the perfect scenario stale quotes will not be executable due to the operator validating them beforehand, however, even though this lowers the severity I believe the vulnerability is still viable. I believe so due to the following scenarios:

1. Operator address is compromised thus operator is acting malicious.
2. Operator makes a mistake/forgets to validate before executing.

For the following reasons I would lower the severity of the vulnerability but I would still keep it. Fully trusting an actor to do the right thing can be dangerous in such cases.

6xiaowu9

Fixed. commit: ad79580(fix: validate new position quote expiry)

Issue M-6: Authorized maker can release collateral from any user's vault, not only their own [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-05-enhanced-may-26th-2026/issues/78>

Summary

`ingressoReleaseCollateralToCustody` lets a maker that has been granted a custody limit move collateral out of the margin pool to a custodian address. Each release request names an arbitrary `request.owner / request.vaultId`, and nothing checks that the signing `maker` is the owner of (or otherwise entitled to) that vault. The per-request limit is computed from the targeted vault's own deposit, so an authorized maker can pull up to the configured basis-point fraction of any user's vault collateral, without the vault owner's consent.

Vulnerability Detail

The release is authorized only by the maker's signature over the request set:

```
uint256 bps = makerCustodyLimitBps[maker][receiver];
if (bps == 0) revert CustodianNotAuthorized();
...
bytes32 digest = _getCustodyReleaseDigest(maker, receiver, nonce, validUntil,
    ↪ requests);
_consumeDigest(digest);
if (!SignatureChecker.isValidSignatureNow(maker, digest, signature)) {
    revert InvalidCustodyReleaseSignature();
}
```

For each request, the code uses `request.owner` directly as the vault owner and never relates it to `maker`.

Impact

A maker that has been granted any non-zero custody limit with a receiver can release up to `bps` percent of the deposited collateral of any vault (including vaults owned by other users) to that receiver. The collateral physically leaves the margin pool while the victim vault's accounting is unchanged, leaving that position under-backed in the pool until the maker voluntarily returns the funds.

Code Snippet

<https://github.com/sherlock-audit/2026-05-enhanced-may-26th-2026/blob/b2e0082f695d5f0a82d8a5051d69423ad4a7cba4/contracts/src/core/EnhancedOptions.sol#L387-L418>

<https://github.com/sherlock-audit/2026-05-enhanced-may-26th-2026/blob/b2e0082f695d5f0a82d8a5051d69423ad4a7cba4/contracts/src/core/EnhancedOptions.sol#L1113-L1134>

Tool Used

Manual Review

Recommendation

Bind each custody-release request to the maker's own vaults. In the per-request loop, require that the maker is the owner of the targeted vault (or maintain an explicit on-chain mapping of which vaults a maker is permitted to borrow against and check it):

```
if (request.owner != maker) revert NotVaultOwner();
```

The protocol team has acknowledged this and stated they will add the necessary ownership checks to restrict releases to vaults the maker is entitled to.

Discussion

6xiaowu9

Fixed in commit 756bf30 (fix: vault maker).

Issue M-7: Unbatched deferred-queue flush bricks cycle advancement [RESOLVED]

Source:
<https://github.com/sherlock-audit/2026-05-enhanced-may-26th-2026/issues/79>

Summary

Users who request a withdraw while a vault is in SETTLED or PROCESSING_DONE are added to an unbounded `_deferredQueueUsers` list. `startNextCycle` flushes the whole list in one loop and re-enqueues every user, at about 46k gas each. Past a few hundred users this exceeds the per-transaction gas cap, so `startNextCycle` can never be included and the vault is stuck in PROCESSING_DONE, unable to start a new cycle. Principal is not lost: the vault can be force-ended via `endVault`, which uses a cheaper flush and lets users reclaim funds.

Vulnerability Detail

The flush cost is linear in the queue size, and the `startNextCycle` path uses the expensive `enqueueUsers = true` branch:

```
function _flushDeferredQueueUsers(bytes32 vaultHash, bool enqueueUsers) internal {
    address[] storage deferredUsers = _deferredQueueUsers[vaultHash];
    uint256 deferredLen = deferredUsers.length;
    for (uint256 i; i < deferredLen; i++) {
        address user = deferredUsers[i];
        _deferredQueued[vaultHash][user] = false;
        if (enqueueUsers) _enqueueUser(vaultHash, user);
        unchecked { ++i; }
    }
    delete _deferredQueueUsers[vaultHash];
}
```

EIP-7825 caps a single transaction at $2^{24} = 16,777,216$ gas. A Foundry PoC measures `startNextCycle` and shows it crosses that cap at roughly 362 deferred users:

Deferred users N	startNextCycle gas	Within 16,777,216 cap
200	9,276,966	yes
300	13,894,566	yes
400	18,512,166	no

Deferred users N	startNextCycle gas	Within 16,777,216 cap
1000	46,217,766	no

The queue grows permissionlessly (one entry per withdraw request), so the threshold is reachable organically on a popular vault or by an actor with a few hundred funded positions. On a chain without EIP-7825 the ceiling is the block gas limit (~30M) and the threshold rises to about 700 users, but the vault still bricks.

endVault flushes the same queue with enqueueUsers = false, so it skips the re-enqueue and stays cheap but still traverses the whole array.

Impact

Once the deferred queue passes the threshold, startNextCycle cannot be included and the vault can never advance to another cycle, with the current cycle's active principal unavailable through the normal flow. There is no permanent fund loss: the owner and operator can force-end the vault via endVault, after which users reclaim principal. So the impact is a permanent halt of the vault's cycle progression plus a temporary principal lock, escapable only by privileged termination.

Code Snippet

- <https://github.com/sherlock-audit/2026-05-enhanced-may-26th-2026/blob/b2e0082f695d5f0a82d8a5051d69423ad4a7cba4/contracts/src/periphery/vault/EnhancedVault.sol#L1643-L1655>
- <https://github.com/sherlock-audit/2026-05-enhanced-may-26th-2026/blob/b2e0082f695d5f0a82d8a5051d69423ad4a7cba4/contracts/src/periphery/vault/EnhancedVault.sol#L1393-L1416>
- <https://github.com/sherlock-audit/2026-05-enhanced-may-26th-2026/blob/b2e0082f695d5f0a82d8a5051d69423ad4a7cba4/contracts/src/periphery/vault/EnhancedVault.sol#L1230-L1247>

Tool Used

Manual Review

Recommendation

Flush the deferred queue in bounded batches with a cursor, so startNextCycle advances it across several transactions and never approaches the per-transaction gas cap.

Discussion

6xiaowu9

Fixed. commit f0fc3a7, 547c5ef(fix: remove deferred queue)

We decided to remove this feature and delete the related variables.

Users will no longer be allowed to perform operations when the vault is not in the `OPEN` state. This resolves the bloated settlement issue, because even if a user performs the operation, it can only be actually completed by `process` after settlement in the next cycle.

cwkang1998

Follow up fixes in latest commits eadd44e, addressing remaining problems raised on this issue.

Issue L-1: Retroactive Change Of `redeemTimePeriod` [ACKNOWLEDGED]

Source:

<https://github.com/sherlock-audit/2026-05-enhanced-may-26th-2026/issues/38>

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

The `ControllerLogic::setRedeemTimePeriod` function allows the owner to retroactively change the redemption time window for all existing and future positions, potentially disadvantaging users with already-open positions.

Vulnerability Detail

The `redeemTimePeriod` variable is a global parameter that defines the time window (in seconds) during which in-the-money (ITM) options can be redeemed after expiry. When the owner calls `setRedeemTimePeriod`, this change is applied immediately to all positions, including those that were opened with different expectations about the redemption deadline.

For example:

- User opens a position expecting a 1-hour redemption window
- Owner subsequently reduces `redeemTimePeriod` to 30 minutes
- User's existing position now has a shortened window, potentially before they can execute their redemption transaction

Conversely, if the period is increased, settlement could be delayed indefinitely for positions that expected an earlier deadline.

Impact

Unfair modification of terms for existing positions. Users may be unable to redeem ITM options within their expected timeframe, or settlement may be delayed unexpectedly. This breaks the implicit contract between users and the protocol regarding position lifecycle parameters.

Code Snippet

```
function setRedeemTimePeriod(uint256 _redeemTimePeriod) external onlyOwner {
    require(_redeemTimePeriod > 0, "C38");
```

```
redeemTimePeriod = _redeemTimePeriod;  
}
```

Tool Used

Manual Review

Recommendation

1. **Add a timelock:** Implement a time-delay mechanism (e.g., 48 hours) before the new `redeemTimePeriod` takes effect, allowing users to close positions under the old terms.
2. **Per-position storage:** Store the `redeemTimePeriod` value at the time each position is opened, then use that stored value when calculating redemption deadlines for that specific position rather than using the global variable.

Discussion

6xiaowu9

Yes, this is a real-time management parameter reserved for special cases of redeem, because novice market makers may forget to redeem or operate slowly, so we need to take precautions for this situation.

cwkang1998

Yes, this is a real-time management parameter reserved for special cases of redeem, because novice market makers may forget to redeem or operate slowly, so we need to take precautions for this situation.

To expand:

This is mainly a function for managing options redemption, which can only be done by the contract owner, which is guarded with a mpc wallet.

The intended use, as mentioned, is to extend an active redemption window when additional processing time is required by market makers. This time period will not be reduced whilst there are active options, and any potential affected parties will be notified in advance.

We acknowledge this as low/informational.

Issue L-2: marginPool address duplicated in EnhancedVault and EnhancedOptions with no synchronization — stale approval after pool migration [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-05-enhanced-may-26th-2026/issues/49>

Summary

EnhancedVault maintains its own marginPool storage variable set via setMarginPool, independently of EnhancedOptions which has its own marginPool variable set via its own setMarginPool. The two are never synchronized. If the owner updates the pool address in one contract but not the other, the vault's token approvals point to a stale address while EnhancedOptions routes funds to the new one, silently breaking settlement.

Vulnerability Detail

EnhancedVault stores and sets marginPool independently:

```
// EnhancedVault.sol:235
address public marginPool;

// EnhancedVault.sol:478-481
function setMarginPool(address _marginPool) external onlyOwner {
    if (_marginPool == address(0)) revert ZeroAddress();
    marginPool = _marginPool;
}
```

EnhancedOptions has its own copy:

```
// EnhancedOptions.sol:47
address public marginPool;

// EnhancedOptions.sol:215-218
function setMarginPool(address _marginPool) external {
    if (_marginPool == ZERO_ADDRESS) revert ZeroAddress();
    marginPool = _marginPool;
}
```

EnhancedOptions exposes no marginPool() getter, so EnhancedVault cannot read the value from the options contract. Both variables must be kept in sync manually.

EnhancedVault uses its local copy to grant token approvals for settlement:

```
// EnhancedVault.sol:487
_forceApprove(_asset, marginPool, _approval ? type(uint256).max : 0);
```

EnhancedOptions uses its own copy to actually move collateral:

```
// EnhancedOptions.sol:440-441
if (ERC20(release.asset).allowance(address(this), marginPool) < amount) {
    _forceApprove(release.asset, marginPool, type(uint256).max);
}
```

A pool migration that updates one but not the other leaves the vault's approvals on the old pool address and grants none to the new one, causing settlement to revert when EnhancedOptions tries to pull collateral through the new pool.

Impact

A pool migration that updates EnhancedOptions.marginPool but not EnhancedVault.marginPool silently breaks the settlement path. All ingressSettle and ingressRedeem calls revert until the owner discovers and corrects the discrepancy. The vault's collateral approvals may also linger on the decommissioned pool address (see Bug 5 for the analogous swap router issue).

Code Snippet

```
// Two independent variables - no link, no sync, no getter
// EnhancedVault.sol
address public marginPool; // set by vault owner

// EnhancedOptions.sol
address public marginPool; // set independently by options owner
```

Tool Used

Manual Review

Recommendation

Add a marginPool() view to IEnhancedOptions and remove the redundant storage variable from EnhancedVault. Have the vault read the address directly from enhancedOptions:

```
// In IEnhancedOptions
function marginPool() external view returns (address);
```

```
// In EnhancedVault - replace local storage with:  
address pool = enhancedOptions.marginPool();  
_forceApprove(_asset, pool, _approval ? type(uint256).max : 0);
```

Discussion

6xiaowu9

Fixed, Commit 9ad9239 fix: Deprecated enhancedVault marginPool

Issue L-3: setSpotShock has no upper bound — owner can instantly liquidate healthy vaults [ACKNOWLEDGED]

Source:

<https://github.com/sherlock-audit/2026-05-enhanced-may-26th-2026/issues/51>

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

setSpotShock enforces only `require(_shockValue > 0)` with no upper bound. The value is read fresh on every `isLiquidatable` and `getExcessCollateral` call. Lowering spotShock increases required margin retroactively for all open naked vaults, allowing the owner to flip healthy vaults into liquidatable states in a single transaction.

Vulnerability Detail

```
// MarginCalculator.sol:268-278
function setSpotShock(..., uint256 _shockValue) external onlyOwner {
    require(_shockValue > 0, "MarginCalculator: invalid spot shock value");
    spotShock[productHash] = _shockValue;
}
```

For a NAKED_PUT vault, the margin formula is:

```
a = min(strike / underlyingPrice, spotShock)
b = max(strike / underlyingPrice - spotShock, 0)
required = (upperBound × a + b) × amount
```

Lowering spotShock from 8e26 to 1e26 for an ATM vault (strike = price = \$2000):

- Old: $a = \min(1, 0.8) = 0.8$, $b = 0.2$, $\text{required} = (0.2 \times 0.8 + 0.2) \times \text{amount} = 0.36 \times \text{amount}$
- New: $a = \min(1, 0.1) = 0.1$, $b = 0.9$, $\text{required} = (0.2 \times 0.1 + 0.9) \times \text{amount} = 0.92 \times \text{amount}$

A vault with 600 USDC collateral writing 1 option at \$2000 strike (originally requiring ~360 USDC) would instantly require ~920 USDC – making it liquidatable in the same block the owner calls setSpotShock.

Impact

Owner can trigger mass liquidations of all healthy naked vaults for a product in a single transaction with no warning and no timelock, front-running vault owners who have no opportunity to add collateral.

Code Snippet

```
function setSpotShock(..., uint256 _shockValue) external onlyOwner {
    require(_shockValue > 0, "MarginCalculator: invalid spot shock value");
    // NO upper bound check
    spotShock[productHash] = _shockValue;
}
```

Tool Used

Manual Review

Recommendation

Add an upper bound check (e.g. `require(_shockValue <= 1e27)`) and implement a timelock (minimum 24 hours) before new `spotShock` values take effect on existing vaults.

Discussion

6xiaowu9

I believe this should be considered invalid / out of scope as a trusted governance parameter risk.

`setSpotShock` is restricted to the `MarginCalculator` owner. There is no permissionless path for an external user to change this value, and the same owner already controls the margin module configuration and upgrade path. Therefore, the described scenario requires the trusted protocol owner/governance to deliberately or incorrectly change a risk parameter.

PeterSRWeb3

It still worth to be in report, it's info vuln.

cwkang1998

To add to this:

We also do not support type 1 vaults, and naked options are not part of the supported protocol flow. This is not reachable for the supported vault types.

We acknowledge this as a low/info issue.

Issue L-4: Oracle returning zero price causes division-by-zero panic, freezing all naked vault operations [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-05-enhanced-may-26th-2026/issues/52>

Summary

`_getNakedMarginRequired` passes the live oracle price directly as the divisor in `FixedPointInt256.div` for `NAKED_PUT` and `COVERED_CALL` formulas. If `oracle.getPrice` returns zero – due to a pricer misconfiguration, validity window expiry, or a bot pushing a zero price – every call to `isLiquidatable`, `getNakedMarginRequired`, and `getExcessCollateral` for affected vaults panics. Undercollateralized vaults cannot be liquidated during the outage.

Vulnerability Detail

```
// MarginCalculator.sol:666
FPI.fromScaledUint(oracle.getPrice(_vaultDetails.shortUnderlyingAsset), BASE);
```

For `NAKED_PUT` and `COVERED_CALL`, the formula divides by `shortUnderlyingPrice`:

```
// MarginCalculator.sol:806-807 (NAKED_PUT)
a = FPI.min(ssd.shortStrike.div(ssd.shortUnderlyingPrice), spotShockValue);
```

`FixedPointInt256.div` computes $(a.value * SCALING_FACTOR) / b.value$. If `b.value = 0`, this is a Solidity 0.8 checked division by zero – an arithmetic panic.

`oracle.getPrice` can return 0 if:

- The pricer's `getPrice()` returns 0
- The `ManualPricer.priceTimeValidity` window has expired (pricer returns stale data)
- The bot hasn't submitted a price since the validity window started

During the outage, `_verifyFinalState` also panics, blocking ALL vault operations including deposit, withdraw, mint.

Impact

A zero oracle price freezes the entire naked margin system. Undercollateralized vaults cannot be liquidated, allowing vault owners to avoid margin calls through price drops while bad debt accumulates unmonitored.

Code Snippet

```
// No zero-check before using oracle price as divisor
FPI.fromScaledUint(oracle.getPrice(_vaultDetails.shortUnderlyingAsset), BASE);
// ...
a = FPI.min(ssd.shortStrike.div(ssd.shortUnderlyingPrice), spotShockValue); // div
↳ by zero if price = 0
```

Tool Used

Manual Review

Recommendation

Add a zero-price guard before using the oracle price:

```
uint256 price = oracle.getPrice(_vaultDetails.shortUnderlyingAsset);
require(price > 0, "MarginCalculator: oracle price is zero");
```

Discussion

6xiaowu9

This issue appears to be already fixed in the latest code.

The current implementation no longer passes `oracle.getPrice()` directly into the naked margin divisor path. `MarginCalculator` now uses `_getNonZeroLivePrice()`, which requires the live oracle price to be greater than zero before constructing `ShortScaledDetails`.

Fixed: commit 1442088 fix: reject zero oracle prices

Issue L-5: Full system pause during Type 2 exercise window permanently locks out buyers [ACKNOWLEDGED]

Source:

<https://github.com/sherlock-audit/2026-05-enhanced-may-26th-2026/issues/53>

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

When the system is fully paused during a Type 2 option's exercise window, `block.timestamp < expiry + redeemTimePeriod` keeps ticking while buyers are blocked from calling `operate()`. The window can expire entirely during the pause, permanently removing buyers' ability to exercise their ITM options. Upon unpauses, buyers are met with a C39 revert while the writer settles and recovers their full collateral.

Vulnerability Detail

The exercise window check is purely time-based:

```
// ControllerLogic.sol:339
require(block.timestamp < expiry + redeemTimePeriod, "C39");
```

The full pause blocks `operate()` entirely:

```
// Controller.sol:381
function operate(Actions.ActionArgs[] memory _actions) external nonReentrant
↳ notFullyPaused {
```

The blockchain clock does not pause. If `fullPauser` pauses the system at `expiry` and unpauses after `redeemTimePeriod` has elapsed, the exercise window has permanently closed. Buyers who held ITM options receive nothing. Writers call `handleSettle` after unpauses and collect their full collateral back as if the option expired OTM.

There is no mechanism to extend `redeemTimePeriod` retroactively for a specific option, and `setRedeemTimePeriod` only affects future options globally.

Impact

A full system pause during a Type 2 exercise window transfers the option's full intrinsic value from buyers to writers with no compensation. Buyers lose both their premium and their ITM payout. There is no on-chain remedy.

Code Snippet

```
// Time keeps ticking during pause - no extension mechanism
require(block.timestamp < expiry + redeemTimePeriod, "C39");

// Full pause blocks all operates including redeem
function operate(...) external nonReentrant notFullyPaused {
```

Tool Used

Manual Review

Recommendation

Track total pause duration per option series and extend `redeemTimePeriod` by the accumulated pause time. Alternatively, record the timestamp when a full pause begins and add the elapsed pause duration to the exercise window deadline when checking C39.

Discussion

6xiaowu9

This is not an issue.

For emergency risk-control situations such as a pause, we should handle them through emergency response measures. For example, we would evaluate whether the redeem window needs to be temporarily extended based on the actual reason for the pause and the actual duration of the pause.

A pause does not necessarily mean that the redeem window must always be extended. A pause itself indicates that some urgent risk situation has occurred, and such risks should be assessed and handled with specific measures tailored to the situation.

cwkang1998

To add to this, `setRedeemTimePeriod` is callable by the privileged owner and is not restricted by the Controller's full-pause state. As such the redemption period can be extended retroactively in times of need.

However, we acknowledge that this is a low/informational issue in extreme conditions.

Issue L-6: setSpotShock, setUpperBoundValues, updateUpperBoundValue take effect immediately on all open vaults — no timelock [ACKNOWLEDGED]

Source:

<https://github.com/sherlock-audit/2026-05-enhanced-may-26th-2026/issues/54>

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

All three margin parameter setters take effect immediately and retroactively affect every open naked vault. There is no timelock, no per-vault snapshot, and no notice period. The owner can increase required margin in a single transaction, making all healthy vaults instantly liquidatable before vault owners can respond.

Vulnerability Detail

These three functions update global storage read fresh on every margin check:

```
// MarginCalculator.sol:276
spotShock[productHash] = _shockValue;

// MarginCalculator.sol:218
maxPriceAtTimeToExpiry[productHash][_timesToExpiry[i]] = _values[i];

// MarginCalculator.sol:254
maxPriceAtTimeToExpiry[productHash][_timeToExpiry] = _value;
```

Every call to `isLiquidatable`, `getExcessCollateral`, and `_verifyFinalState` reads these values live. The moment the owner updates them, all open naked vaults for that product recalculate margin at the new parameters.

Attack scenario: Owner raises `spotShock` from `0.5e27` to `0.99e27` for ETH puts. A vault with 600 USDC backing 1 ETH put at \$2000 strike (ETH at \$2100):

- Old required: ~300 USDC → vault healthy
- New required: ~1900 USDC → vault instantly liquidatable

In the same block, owner or a colluding liquidator liquidates all affected vaults.

Impact

Owner can trigger mass liquidation of all healthy naked vaults for any product in a single transaction with no warning. Vault owners have zero opportunity to add collateral. This

is a complete undermining of the margin safety model.

Code Snippet

```
// Reads live on every margin check - no per-vault snapshot
FPI.FixedPointInt memory spotShockValue =
    ↪ FPI.FixedPointInt(int256(spotShock[_productHash]));
FPI.FixedPointInt memory optionUpperBoundValue = _findUpperBoundValue(_productHash,
    ↪ _shortExpiryTimestamp);
```

Tool Used

Manual Review

Recommendation

Implement a two-step update with a minimum 24-hour timelock for `setSpotShock`, `setUpperBoundValues`, and `updateUpperBoundValue`. New values should only take effect after the timelock expires, giving vault owners time to adjust collateral.

Discussion

6xiaowu9

I believe this should be invalid / out of scope as stated. #51

The only caller able to update these parameters is the trusted `MarginCalculator` owner/governance. There is no permissionless path for an external user to change `spotShock` or upper-bound values, and the same governance already controls the margin module configuration and upgrade path. This is therefore a governance parameter-management risk, not an untrusted-user exploit.

cwkang1998

To add to this: Similar to #51, this is also outside of our supported protocol flow.

We acknowledge this as a low/information issue.

Issue L-7: Insolvent expired naked vault permanently drains shared pool and blocks solvent writers [ACKNOWLEDGED]

Source:

<https://github.com/sherlock-audit/2026-05-enhanced-may-26th-2026/issues/55>

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

When a Type-1 naked vault expires deeply ITM without being liquidated, `handleRedeem` pays the buyer's full payout from the shared `MarginPool` with no vault solvency check – consuming other writers' collateral to cover the shortfall. The insolvent vault permanently blocks settlement via `getExcessCollateral` reverting with "FixedPointInt256: negative value", stranding the writer's collateral forever and leaving the last solvent writer unable to recover their own funds when the pool is drained.

Vulnerability Detail

`handleRedeem` has no vault solvency check – it pays from the shared pool based purely on oracle prices:

```
// ControllerLogic.sol:370-373
uint256 payout = getPayout(_args.otoken, _args.amount);
otoken.burnOtoken(sender, _args.amount);
pool.transferToUser(collateral, _args.receiver, payout); // draws from shared
↳ pool, no vault check
```

`MarginPool` uses a single global balance with no per-vault isolation:

```
// MarginPool.sol:120
assetBalance[_asset] = assetBalance[_asset] - _amount;
```

When the insolvent writer tries to settle, `getExcessCollateral` computes `collateral(1000) - required(1800) = -800` in FPI, then:

```
// FixedPointInt256.sol:68
require(_a.value >= 0, "FixedPointInt256: negative value"); // REVERT
```

The vault can never be settled. The writer's 1000 USDC is permanently stranded. The 800 USDC deficit was already paid from other writers' collateral.

Impact

High. An insolvent naked vault that expires without liquidation causes: (1) buyer paid from shared pool consuming other writers' collateral; (2) insolvent vault permanently unseizable, collateral stranded; (3) last solvent writer cannot recover legitimate collateral – pool arithmetic underflow. No on-chain resolution path exists.

Code Snippet

```
// handleRedeem - no vault solvency check
uint256 payout = getPayout(_args.otoken, _args.amount);
pool.transferToUser(collateral, _args.receiver, payout);

// getExcessCollateral - reverts on negative balance
FPI.FixedPointInt memory excessCollateral =
    ↪ collateralAmount.sub(collateralRequired);
uint256 excessCollateralExternal =
    ↪ excessCollateral.toScaledUint(collateralDecimals, isExcess);
// ^ reverts with "FixedPointInt256: negative value" when insolvent
```

Tool Used

Manual Review

Recommendation

Add a per-vault solvency check in `handleRedeem` to cap payout at the vault's actual collateral, or isolate pool balances per-vault so bad debt from one vault cannot consume another vault's collateral.

Discussion

6xiaowu9

I believe this issue is invalid as stated.

In the latest code, `oTokens` are not shared globally only by product parameters. The `oToken` identity includes `vaultOwner`, and `EnhancedOptions` creates the `oToken` with `vaultOwner = sellerConfirmation.taker`. Therefore different makers/users do not mint into the same cash-settled `oToken` address.

Additionally, redemption is not a permissionless path. `ingressRedeem` is operator-gated, and the backend/operator performs off-chain position and balance checks before signing/submitting the redeem action. The shared `MarginPool.assetBalance` is the custody pool balance, not the full per-user/per-position accounting source of truth.

Given this trust model, the report's scenario where an arbitrary insolvent writer's option holder drains unrelated writers through a shared pool is not reachable as described. Any redeem executed by the operator is expected to be bounded by the backend's per-position accounting for that `vaultOwner/oToken`.

At most, a defense-in-depth improvement would be to mirror this backend accounting on-chain with per-oToken or per-vault reserve tracking. But the current report assumes a permissionless redeem path and globally shared oToken series, neither of which matches the latest code.

PeterSRWeb3

I believe this issue is invalid as stated.

In the latest code, oTokens are not shared globally only by product parameters. The oToken identity includes `vaultOwner`, and `EnhancedOptions` creates the oToken with `vaultOwner = sellerConfirmation.taker`. Therefore different makers/users do not mint into the same cash-settled oToken address.

Additionally, redemption is not a permissionless path. `ingressRedeem` is operator-gated, and the backend/operator performs off-chain position and balance checks before signing/submitting the redeem action. The shared `MarginPool.assetBalance` is the custody pool balance, not the full per-user/per-position accounting source of truth.

Given this trust model, the report's scenario where an arbitrary insolvent writer's option holder drains unrelated writers through a shared pool is not reachable as described. Any redeem executed by the operator is expected to be bounded by the backend's per-position accounting for that `vaultOwner/oToken`.

At most, a defense-in-depth improvement would be to mirror this backend accounting on-chain with per-oToken or per-vault reserve tracking. But the current report assumes a permissionless redeem path and globally shared oToken series, neither of which matches the latest code.

Agreed that the cross-writer drain via shared oToken series was overstated – that part of the original description was imprecise and I will rewrite it.

The bug that remains has nothing to do with redeem, oToken identity, or the operator trust model. It is purely an accounting invariant violation: `nakedPoolBalance` is incremented when a naked vault deposits collateral (`handleDepositCollateral:201`) but is never decremented for an insolvent expired vault, because every decrement path is blocked simultaneously at expiry – settlement reverts at C32 before reaching line 519, liquidation reverts at "can not liquidate expired position" in `MarginCalculator`, and withdrawal reverts at C22. No operator action, admin call, or off-chain check can decrement `nakedPoolBalance` – it is a mapping only writable by `ControllerLogic` and there is no escape hatch for this state.

The most direct evidence this is unintentional: the code already handles the exact same problem for `typeVault==2` at lines 428-431 with an explicit insolvent-vault carve-out that forces settlement through. `typeVault==1` has no equivalent, which means a single insolvent expired naked vault permanently inflates the cap counter, causing all subsequent naked deposits on that collateral asset to revert at C37 – regardless of the

operator's off-chain accounting and regardless of whether anyone ever calls redeem.

6xiaowu9

Thanks for the clarification. We agree that, if a type-1 vault already existed, an expired insolvent vault would have no successful cleanup path and its `nakedPoolBalance` contribution could remain recorded. However, this state is not reachable through the current protocol: `EnhancedOptions._executeNewPosition` creates only type 0 or type 2 vaults; it never creates type 1. Every vault-mutating Controller action requires `msg.sender == manager`. The configured manager is `EnhancedOptions`, which exposes no generic entrypoint allowing users or operators to submit an arbitrary type-1 `OpenVault` action. Therefore, no user can create the naked vault required by this report, and no existing supported flow can produce the claimed C37 denial of service. The type-2 carve-out is also not an equivalent precedent. Type-2 settlement is allowed only after `expiry + redeemTimePeriod`, when C39 permanently prevents further exercises. Cash-settled type-1 oTokens have no equivalent redemption deadline, so blindly applying the same carve-out could close accounting while holder liabilities remain redeemable. We consequently consider this a latent defect in currently unused type-1 functionality, rather than a reachable vulnerability in the deployed protocol. It must be addressed before type-1 vaults are enabled, but it does not establish impact under the current configuration.

PeterSRWeb3

Agreed. You are correct that no current flow produces a type-1 vault, and with `setOperatorsEnabled` permanently gated by `require(!_enabled)`, there is no live exploit path. The original cross-writer drain framing was imprecise and the reachability defense holds under the current deployment.

The accounting defect itself is real – `nakedPoolBalance` has no decrement path for an insolvent expired type-1 vault, and the `typeVault==2` carve-out at lines 428–431 confirms the developers already solved this for one vault type. The fix (an owner-callable escape hatch that zeroes the counter for a stuck vault, without touching payouts) should be applied before type-1 is enabled.

I think this can be left as **Low**: a confirmed latent defect in currently unused functionality, no live exploit path, must be resolved before type-1 vaults are activated.

cwkang1998

We acknowledge this as a low/informational issue.

Issue L-8: setOperatorsEnabled Is Unusable And Redundant [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-05-enhanced-may-26th-2026/issues/56>

Summary

The `setOperatorsEnabled` function contains a logical constraint that prevents the operators feature from ever being activated, rendering the entire operator authorization system non-functional.

Vulnerability Detail

The `setOperatorsEnabled` function is designed to toggle the `operatorsEnabled` state variable, which controls whether vault owners can authorize operators to act on their behalf. However, the function contains a `require` statement that explicitly prevents this:

```
function setOperatorsEnabled(bool _enabled) external onlyOwner {
    require(!_enabled, "operators cannot be enabled");
    operatorsEnabled = _enabled;
}
```

The `require(!_enabled, "operators cannot be enabled")` check forces the function to only succeed when `_enabled` is false. Since `operatorsEnabled` initializes as false in the state variables, this function can only ever set it to false again—it can never transition to true. This creates an invariant where `operatorsEnabled` is permanently locked at false.

Impact

The operators feature is completely disabled and unusable:

1. While vault owners can still call `setOperator` to designate operators in the operators mapping, these operators cannot perform any vault actions
2. The authorization logic in `_isAuthorized` requires `operatorsEnabled` to be true for operators to gain any privileges:

```
require(
    (_sender == manager) || (operators[_accountOwner][_sender] && operatorsEnabled)
    || (_sender == _accountOwner && operatorsEnabled),
    "C6"
);
```

1. Since `operatorsEnabled` is always false, only the manager address can ever execute vault actions; vault owners themselves cannot execute actions if operators are intended to be enabled
2. The entire operator-based authorization system is dead code that provides false sense of functionality

Code Snippet

```
function setOperatorsEnabled(bool _enabled) external onlyOwner {  
    require(!_enabled, "operators cannot be enabled");  
    operatorsEnabled = _enabled;  
}
```

Tool Used

Manual Review

Recommendation

Remove the `require(!_enabled, "operators cannot be enabled")` constraint to allow the `operatorsEnabled` flag to be toggled:

```
function setOperatorsEnabled(bool _enabled) external onlyOwner {  
    require(operatorsEnabled != _enabled, "C9");  
    operatorsEnabled = _enabled;  
}
```

Alternatively, if operators should remain permanently disabled, remove the entire `setOperatorsEnabled` function and the `operatorsEnabled` state variable along with all code paths that depend on it to reduce complexity and eliminate dead code.

Discussion

6xiaowu9

Fixed: <https://github.com/sherlock-audit/2026-05-enhanced-may-26th-2026/pull/3>
[allowbreak #discussion_r3371487856](#)

Issue L-9: `_retrieveOtokenForFlashLoanRedeem` Function Is Currently Not Used And Should Be Removed [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-05-enhanced-may-26th-2026/issues/59>

Summary

An internal function `_retrieveOtokenForFlashLoanRedeem` exists in `EnhancedOptions` in the codebase but is only referenced in commented-out code, making it dead code with no actual usage in the active contract.

Vulnerability Detail

The function `_retrieveOtokenForFlashLoanRedeem` is defined as an internal helper function but is never called by any active, non-commented code:

```
function _retrieveOtokenForFlashLoanRedeem(address redeemer, address otoken,
↪ uint256 amount) internal {
    // create operation to withdraw otoken from mmarket
    MMarketOperations.Operation[] memory operationsArray = new
    ↪ MMarketOperations.Operation[](1);
    MMarketOperations.Operation memory withdrawOperation =
    ↪ MMarketOperations.Operation(
        MMarketOperations.OperationType.Withdraw,
        redeemer, // user to withdraw from
        address(this), // withdraw to this address
        otoken, // otoken address
        address(0), // asset_2 not needed
        amount, // amount of otoken to withdraw
        0, // amount_2 not needed
        bytes("0")
    );
    operationsArray[0] = withdrawOperation;

    mmarket.operate(operationsArray);
}
```

This function is only invoked in the `executeOperation` function, which is entirely commented out (lines 761-813). The commented code relates to flash loan redeem functionality that appears to be either under development, disabled, or removed:

```
// /**
```

```
// * @notice Callback from flash loan provider. Executes a physical option
→ redemption using loaned funds, then swaps collateral back into borrowed asset
→ to repay loan.
// */
// function executeOperation(
//     address asset,
//     uint256 amount,
//     uint256 premium,
//     address initiator,
//     bytes calldata params
// )
//     external
//     nonReentrant
//     returns (bool)
// {
//     // ...
//     _retrieveOtokenForFlashLoanRedeem(redeemer, args.asset, args.amount); //
→ <-- ONLY CALL LOCATION
//     // ...
// }
```

Impact

1. **Code Bloat:** Dead code increases contract size unnecessarily, consuming deployment gas and making the codebase harder to maintain.
2. **Confusion:** Developers reviewing the code may waste time analyzing a function that isn't actually used, or assume functionality exists when it's disabled.
3. **Maintenance Burden:** Dead code creates false dependency tracking. If this function were ever modified for bug fixes, the changes would have no effect since nothing calls it.
4. **Audit Surface:** Auditors must review this code even though it doesn't execute, increasing audit scope and cost without security benefit.
5. **Deployment Risk:** If the commented `executeOperation` function were accidentally uncommented without proper review, dead code that seems functional could mask actual issues.

Code Snippet

`_retrieveOtokenForFlashLoanRedeem:`

```
function _retrieveOtokenForFlashLoanRedeem(address redeemer, address otoken,
→ uint256 amount) internal {
    // create operation to withdraw otoken from mmarket
    MMarketOperations.Operation[] memory operationsArray = new
→ MMarketOperations.Operation[](1);
```

```

MMarketOperations.Operation memory withdrawOperation =
↳ MMarketOperations.Operation(
    MMarketOperations.OperationType.Withdraw,
    redeemer, // user to withdraw from
    address(this), // withdraw to this address
    otoken, // otoken address
    address(0), // asset_2 not needed
    amount, // amount of otoken to withdraw
    0, // amount_2 not needed
    bytes("0")
);
operationsArray[0] = withdrawOperation;

mmarket.operate(operationsArray);
}

```

Tool Used

Manual Review

Recommendation

Remove the `_retrieveOtokenForFlashLoanRedeem` function entirely:

```

// DELETE THIS ENTIRE FUNCTION:
// function _retrieveOtokenForFlashLoanRedeem(address redeemer, address otoken,
↳ uint256 amount) internal {
//     MMarketOperations.Operation[] memory operationsArray = new
↳ MMarketOperations.Operation[](1);
//     MMarketOperations.Operation memory withdrawOperation =
↳ MMarketOperations.Operation(
//         MMarketOperations.OperationType.Withdraw,
//         redeemer,
//         address(this),
//         otoken,
//         address(0),
//         amount,
//         0,
//         bytes("0")
//     );
//     operationsArray[0] = withdrawOperation;
//     mmarket.operate(operationsArray);
// }

```

Discussion

6xiaowu9

Fixed. commit f83f663(chore: remove unused flash loan redeem code)

Issue L-10: `_constructFlashLoanRedeemActionArgs` Function Is Currently Not Used And Should Be Removed [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-05-enhanced-may-26th-2026/issues/60>

Summary

A helper function `_constructFlashLoanRedeemActionArgs` in the Actions library exists but is never called by any active, non-commented code, making it dead code that serves no purpose in the current contract.

Vulnerability Detail

The function `_constructFlashLoanRedeemActionArgs` is defined in the Actions library but is only referenced in commented-out code within `EnhancedOptions`:

```
function _constructFlashLoanRedeemActionArgs(bytes calldata packedEncodedArgs)
    internal
    view
    returns (ActionArgs memory, address, bytes memory, uint256)
{
    uint256 len;
    address otoken;
    uint256 amount;
    address redeemer;
    uint256 maxAmountIn;
    bytes memory uniswapRoute;

    len = packedEncodedArgs.length;
    otoken = address(bytes20(packedEncodedArgs[0:20]));
    amount = uint256(bytes32(packedEncodedArgs[20:52]));
    redeemer = address(bytes20(packedEncodedArgs[52:72]));
    maxAmountIn = uint256(bytes32(packedEncodedArgs[72:104]));
    uniswapRoute = packedEncodedArgs[104:len];

    return (
        ActionArgs({
            actionType: Actions.ActionType.Redeem,
            owner: address(0),
            secondAddress: address(this),
            asset: otoken,
            vaultId: 0,
            amount: amount,
```

```

        index: 0,
        data: bytes("")
    }),
    redeemer,
    uniswapRoute,
    maxAmountIn
);
}

```

This function's only caller is in the commented-out `executeOperation` function in `Enhanced Options`:

```

// function executeOperation(
//     address asset,
//     uint256 amount,
//     uint256 premium,
//     address initiator,
//     bytes calldata params
// ) external nonReentrant returns (bool) {
//     // ...
//     (Actions.ActionArgs memory args, address redeemer, bytes memory swapRoute,
//     ↪ uint256 amountInMaximum) =
//     ↪ Actions._constructFlashLoanRedeemActionArgs(params); // <-- ONLY CALL
//     ↪ LOCATION (COMMENTED)
//     // ...
// }

```

Since the calling function is entirely commented out, `_constructFlashLoanRedeemActionArgs` has zero active execution paths.

Impact

1. **Unnecessary Library Bloat:** Dead utility functions increase library bytecode size, raising gas costs for every contract that imports the `Actions` library. This affects not just `EnhancedOptions` but potentially other contracts using the `Actions` library.
2. **Maintenance Confusion:** Developers may assume this function is active and try to use it, only to discover it's dead code. This wastes time and creates false expectations about available functionality.
3. **Audit Complexity:** Auditors must review and reason about code that never executes, increasing scope and complexity without actual security benefit.

Code Snippet

`_constructFlashLoanRedeemActionArgs:`

```

function _constructFlashLoanRedeemActionArgs(bytes calldata packedEncodedArgs)
    internal
    view
    returns (ActionArgs memory, address, bytes memory, uint256)
{
    uint256 len;
    address otoken;
    uint256 amount;
    address redeemer;
    uint256 maxAmountIn;
    bytes memory uniswapRoute;

    len = packedEncodedArgs.length;
    otoken = address(bytes20(packedEncodedArgs[0:20]));
    amount = uint256(bytes32(packedEncodedArgs[20:52]));
    redeemer = address(bytes20(packedEncodedArgs[52:72]));
    maxAmountIn = uint256(bytes32(packedEncodedArgs[72:104]));
    uniswapRoute = packedEncodedArgs[104:len];

    return (
        ActionArgs({
            actionType: Actions.ActionType.Redeem,
            owner: address(0),
            secondAddress: address(this),
            asset: otoken,
            vaultId: 0,
            amount: amount,
            index: 0,
            data: bytes("")
        }),
        redeemer,
        uniswapRoute,
        maxAmountIn
    );
}

```

Tool Used

Manual Review

Recommendation

Remove the `_constructFlashLoanRedeemActionArgs` function entirely from the Actions library:

```
// DELETE THIS ENTIRE FUNCTION:
// function _constructFlashLoanRedeemActionArgs(bytes calldata packedEncodedArgs)
//     internal
//     view
//     returns (ActionArgs memory, address, bytes memory, uint256)
// {
//     // ...function body...
// }
```

Discussion

6xiaowu9

Fixed. commit f83f663(chore: remove unused flash loan redeem code)

Issue L-11: priceTimeValidity Parameter Is Globally Mutable And Retroactive [ACKNOWLEDGED]

Source:

<https://github.com/sherlock-audit/2026-05-enhanced-may-26th-2026/issues/61>

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

The `priceTimeValidity` parameter is globally mutable and retroactively affects the validity window of all previously set prices, allowing the owner to arbitrarily invalidate or revalidate historical prices by changing this parameter.

Vulnerability Detail

The `ManualPricer` contract maintains a single global `priceTimeValidity` parameter that defines how long prices remain valid after being set:

```
function setPriceTimeValidity(uint256 _priceTimeValidity) external onlyOwner {
    require(_priceTimeValidity > 0, "ManualPricer: price time validity cannot be
    ↳ 0");
    priceTimeValidity = _priceTimeValidity;
}
```

This parameter is then used unconditionally when checking price freshness in `getPrice()`:

```
function getPrice() external view override returns (uint256) {
    require(block.timestamp <= lastExpiryTimestamp + priceTimeValidity,
    ↳ "ManualPricer: price is no longer valid");
    return historicalPrice[lastExpiryTimestamp];
}
```

The problem is that changing `priceTimeValidity` retroactively affects the validity of ALL previously set prices, not just future prices.

Example scenario:

- Time = 1000: Bot sets price with `priceTimeValidity = 3600` seconds (1 hour)
 - Price valid until: $1000 + 3600 = 4600$
- Time = 4500: Owner calls `setPriceTimeValidity(300)` (change to 5 minutes)
- Time = 4500: Price checked by `getPrice()`
 - New check: $4500 \leq 1000 + 300 \rightarrow 4500 \leq 1300 \rightarrow \text{FALSE}$
 - Price is now considered **invalid** even though it was set to be valid for 1 hour

Conversely, if the owner increases the validity window:

- Time = 1500: Bot sets price with `priceTimeValidity = 300`
 - Price would normally be valid until: $1500 + 300 = 1800$
- Time = 1750: Price is considered invalid ($750 > 300$)
- Time = 1750: Owner calls `setPriceTimeValidity(3600)`
- Time = 1750: Price checked by `getPrice()`
 - New check: $1750 \leq 1500 + 3600 \rightarrow 1750 \leq 5100 \rightarrow \text{TRUE}$
 - Price is now considered **valid** after being invalid

Impact

1. **Retroactive Validity Changes:** The owner can arbitrarily invalidate prices that were previously valid by lowering `priceTimeValidity`, disrupting services that depend on price availability.
2. **Price Resurrection:** The owner can retroactively revalidate expired prices by raising `priceTimeValidity`, potentially resurrecting stale pricing data that should have expired.
3. **Broken Price Validity Guarantees:** Contracts relying on this pricer cannot make assumptions about price validity windows. A price that was guaranteed valid for X seconds can suddenly become invalid based on owner actions.

Code Snippet

`setPriceTimeValidity:`

```
function setPriceTimeValidity(uint256 _priceTimeValidity) external onlyOwner {
    require(_priceTimeValidity > 0, "ManualPricer: price time validity cannot be
    ↪ 0");
    priceTimeValidity = _priceTimeValidity;
}
```

`getPrice` - uses global `priceTimeValidity`:

```
function getPrice() external view override returns (uint256) {
    require(block.timestamp <= lastExpiryTimestamp + priceTimeValidity,
    ↪ "ManualPricer: price is no longer valid");
    return historicalPrice[lastExpiryTimestamp];
}
```

Tool Used

Manual Review

Recommendation

Store the validity duration at the time each price is set, so each price has an immutable validity window. Modify the contract to track both the price and its associated validity duration:

```
/// @notice mapping of timestamp to price and its validity window
mapping(uint256 => PriceData) public historicalPriceData;

struct PriceData {
    uint256 price;
    uint256 validityWindow; // stored at time of setting
}

/**
 * @notice set the expiry price in the oracle, can only be called by Bot address
 * @param _expiryTimestamp expiry to set a price for
 * @param _price price of the asset in USD, scaled by 1e8
 */
function setExpiryPriceInOracle(uint256 _expiryTimestamp, uint256 _price) external
    ↪ onlyBot {
    require(_expiryTimestamp <= block.timestamp, "ManualPricer: expiries prices
    ↪ cannot be set for the future");

    uint256 previousPrice = historicalPriceData[lastExpiryTimestamp].price;

    if (previousPrice > 0) {
        require(
            _price * MULTIPLIER_FACTOR < previousPrice * deviationMultiplier
            && _price * deviationMultiplier > previousPrice * MULTIPLIER_FACTOR,
            "ManualPricer: price deviation is larger than currently allowed"
        );
    }

    lastExpiryTimestamp = _expiryTimestamp;
    // Store price WITH its validity window at the time of setting
    historicalPriceData[_expiryTimestamp] = PriceData({
        price: _price,
        validityWindow: priceTimeValidity // Capture current validity at time of
        ↪ setting
    });
    oracle.setExpiryPrice(asset, _expiryTimestamp, _price);
}

/**
```

```

* @notice get the live price for the asset
* @dev overrides the getPrice function in EnhancedPricerInterface
* @return price of the asset in USD, scaled by 1e8
*/
function getPrice() external view override returns (uint256) {
    PriceData storage priceData = historicalPriceData[lastExpiryTimestamp];
    require(
        block.timestamp <= lastExpiryTimestamp + priceData.validityWindow,
        "ManualPricer: price is no longer valid"
    );
    return priceData.price;
}

```

This ensures that:

- Each price is locked to its validity window at the time it was set
- Changing `priceTimeValidity` only affects **future** prices, not historical ones
- Price validity is deterministic and immutable once set
- The contract maintains a full audit trail of what validity window applied to each price

Alternatively, if immutable validity windows per price are too restrictive, document the behavior clearly and add events that emit when `priceTimeValidity` changes, allowing external systems to react to these changes.

Discussion

6xiaowu9

We believe this issue should be considered invalid as a security vulnerability, or informational at most. The reported state transition is technically possible, but `priceTimeValidity` is a global runtime freshness policy controlled exclusively by the trusted `ManualPricer` owner. It is not represented or documented as an immutable per-price commitment. Updating it is therefore expected to immediately change the freshness policy applied by `getPrice()`. Furthermore, the parameter only affects the latest live price returned by `ManualPricer.getPrice()`. It does not modify or revalidate historical expiry prices already stored in `Oracle.storedPrice`, which are retrieved independently through `Oracle.getExpiryPrice()`. `ManualPricer.getHistoricalPrice()` is deprecated. Therefore, the claim that all historical prices are retroactively modified or resurrected is inaccurate. Decreasing the value causes the live-price path to fail closed, while increasing it allows the latest submitted price under the newly selected governance policy. Both scenarios require an authorized governance action; there is no permissionless attack path, privilege escalation, or demonstrated violation of an immutable validity guarantee. The same owner also controls the UUPS upgrade authorization, so the report describes trusted governance/configuration risk rather than an external exploit. No code change is required.

zyppyvids

We believe this issue should be considered invalid as a security vulnerability, or informational at most. The reported state transition is technically possible, but price-TimeValidity is a global runtime freshness policy controlled exclusively by the trusted ManualPricer owner. It is not represented or documented as an immutable per-price commitment. Updating it is therefore expected to immediately change the freshness policy applied by getPrice(). Furthermore, the parameter only affects the latest live price returned by ManualPricer.getPrice(). It does not modify or revalidate historical expiry prices already stored in Oracle.storedPrice, which are retrieved independently through Oracle.getExpiryPrice(). ManualPricer.getHistoricalPrice() is deprecated. Therefore, the claim that all historical prices are retroactively modified or resurrected is inaccurate. Decreasing the value causes the live-price path to fail closed, while increasing it allows the latest submitted price under the newly selected governance policy. Both scenarios require an authorized governance action; there is no permissionless attack path, privilege escalation, or demonstrated violation of an immutable validity guarantee. The same owner also controls the UUPS upgrade authorization, so the report describes trusted governance/configuration risk rather than an external exploit. No code change is required.

I believe it should be marked as acknowledged - severity of Low/Informational is the right one.

Issue L-12: No Enforcement That New Expiry Timestamp > lastExpiryTimestamp [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-05-enhanced-may-26th-2026/issues/62>

Summary

The bot can retroactively change the contract's "current price" by setting prices for past timestamps without any chronological ordering enforcement, allowing manipulation of which historical price is treated as the active price.

Vulnerability Detail

The `setExpiryPriceInOracle` function in `ManualPricer` accepts any timestamp in the past (or present) and unconditionally updates `lastExpiryTimestamp` to that value without enforcing that new timestamps are chronologically newer:

```
function setExpiryPriceInOracle(uint256 _expiryTimestamp, uint256 _price) external
↳ onlyBot {
    require(_expiryTimestamp <= block.timestamp, "ManualPricer: expiries prices
↳ cannot be set for the future");

    uint256 previousPrice = historicalPrice[lastExpiryTimestamp];

    if (previousPrice > 0) {
        require(
            _price * MULTIPLIER_FACTOR < previousPrice * deviationMultiplier
            && _price * deviationMultiplier > previousPrice * MULTIPLIER_FACTOR,
            "ManualPricer: price deviation is larger than currently allowed"
        );
    }

    lastExpiryTimestamp = _expiryTimestamp; // NO CHECK: could be older than
↳ previous lastExpiryTimestamp
    historicalPrice[_expiryTimestamp] = _price;
    oracle.setExpiryPrice(asset, _expiryTimestamp, _price);
}
```

The `getPrice` function treats `lastExpiryTimestamp` as the "current price" and validates freshness based on when that timestamp occurred:

```
function getPrice() external view override returns (uint256) {
    require(block.timestamp <= lastExpiryTimestamp + priceTimeValidity,
↳ "ManualPricer: price is no longer valid");
}
```

```
    return historicalPrice[lastExpiryTimestamp];  
}
```

Attack scenario:

Timeline:

- **Block 1000:** Bot calls `setExpiryPriceInOracle(1000, 100)`
 - `lastExpiryTimestamp = 1000`
 - `historicalPrice[1000] = 100`
- **Block 5000:** Application calls `getPrice()`
 - Check: $5000 \leq 1000 + \text{priceTimeValidity}$
 - If `priceTimeValidity = 3600`: $5000 \leq 4600 \rightarrow$ **FAILS (price expired)**
 - Price cannot be retrieved
- **Block 5000:** Bot calls `setExpiryPriceInOracle(4000, 110)` (setting a price for an older time)
 - `lastExpiryTimestamp = 4000` (changed to an older timestamp!)
 - `historicalPrice[4000] = 110`
- **Block 5000:** Application calls `getPrice()` again
 - Check: $5000 \leq 4000 + 3600$
 - $5000 \leq 7600 \rightarrow$ **PASSES (price now valid!)**
 - Returns `historicalPrice[4000] = 110`

The bot has retroactively resurrected an expired price by setting `lastExpiryTimestamp` to a newer timestamp, extending the freshness window.

Another scenario - regression:

- **Block 1000:** Bot sets price 100 at timestamp 1000
- **Block 2000:** Bot sets price 105 at timestamp 2000 (chronologically correct)
- **Block 2500:** Bot calls `setExpiryPriceInOracle(1500, 102)` (retroactively sets older price)
- `lastExpiryTimestamp` regresses from 2000 to 1500
- The "current price" is now from 500 blocks ago instead of 500 blocks ago... but worse, it broke chronological ordering

Impact

1. **Price Manipulation:** The bot can retroactively change which historical price is considered "current" by setting prices for past timestamps, bypassing intended

price freshness guarantees.

2. **Replay of Expired Prices:** Once a price expires, the bot can resurrect it by setting a price to a newer historical timestamp, extending its validity window unexpectedly.
3. **Chronological Chaos:** There's no guarantee that `lastExpiryTimestamp` moves forward in time. The bot can cause `lastExpiryTimestamp` to regress, meaning the "current price" can be from an arbitrarily old timestamp.
4. **Broken Price Freshness Model:** Consumers of `getPrice()` expect to get a recently-set price, but the bot can cause old prices to be returned as "current" simply by not setting newer prices and allowing time to pass.

Code Snippet

`setExpiryPriceInOracle:`

```
function setExpiryPriceInOracle(uint256 _expiryTimestamp, uint256 _price) external
↳ onlyBot {
    require(_expiryTimestamp <= block.timestamp, "ManualPricer: expiries prices
↳ cannot be set for the future");

    uint256 previousPrice = historicalPrice[lastExpiryTimestamp];

    if (previousPrice > 0) {
        require(
            _price * MULTIPLIER_FACTOR < previousPrice * deviationMultiplier
            && _price * deviationMultiplier > previousPrice * MULTIPLIER_FACTOR,
            "ManualPricer: price deviation is larger than currently allowed"
        );
    }

    lastExpiryTimestamp = _expiryTimestamp; // NO CHECK: could be older than
↳ previous lastExpiryTimestamp
    historicalPrice[_expiryTimestamp] = _price;
    oracle.setExpiryPrice(asset, _expiryTimestamp, _price);
}
```

Tool Used

Manual Review

Recommendation

Enforce that new expiry timestamps are strictly greater than the previous `lastExpiryTimestamp` to maintain chronological ordering:

```

function setExpiryPriceInOracle(uint256 _expiryTimestamp, uint256 _price) external
↳ onlyBot {
    require(_expiryTimestamp <= block.timestamp, "ManualPricer: expiries prices
↳ cannot be set for the future");

    // ADD THIS CHECK
    require(_expiryTimestamp > lastExpiryTimestamp, "ManualPricer: new expiry must
↳ be after last expiry");

    uint256 previousPrice = historicalPrice[lastExpiryTimestamp];

    if (previousPrice > 0) {
        require(
            _price * MULTIPLIER_FACTOR < previousPrice * deviationMultiplier
            && _price * deviationMultiplier > previousPrice * MULTIPLIER_FACTOR,
            "ManualPricer: price deviation is larger than currently allowed"
        );
    }

    lastExpiryTimestamp = _expiryTimestamp;
    historicalPrice[_expiryTimestamp] = _price;
    oracle.setExpiryPrice(asset, _expiryTimestamp, _price);
}

```

This ensures:

- Prices can only move forward in time, never backward
- lastExpiryTimestamp monotonically increases with each call
- The "current price" is always from a more recent timestamp than the previous one
- Price expiration becomes deterministic and cannot be manipulated retroactively

Discussion

6xiaowu9

Fixed. The current commit d1e9d04 adds `_expiryTimestamp > lastExpiryTimestamp` in `setExpiryPriceInOracle`, while retaining the check that future timestamps cannot be used.

Issue L-13: Unused addressbook Declaration [ACKNOWLEDGED]

Source:

<https://github.com/sherlock-audit/2026-05-enhanced-may-26th-2026/issues/63>

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

The addressbook state variable in ManualPricer is declared, initialized, and stored on-chain but is never read or used anywhere in the contract, representing unnecessary storage bloat and initialization gas cost.

Vulnerability Detail

The addressbook state variable is declared as a public storage variable:

```
/// @notice the enhanced addressbook address
AddressBookInterface public addressbook;
```

It is validated and initialized during contract setup:

```
function initialize(address _bot, address _asset, address _oracle, address
↪ _addressbook, address _owner)
    public
    initializer
{
    require(_bot != address(0), "ManualPricer: Cannot set 0 address as bot");
    require(_oracle != address(0), "ManualPricer: Cannot set 0 address as oracle");
    require(_addressbook != address(0), "ManualPricer: Cannot set 0 address as
↪ addressbook");
    __Ownable_init(_owner);
    bot = _bot;
    oracle = OracleInterface(_oracle);
    asset = _asset;
    addressbook = AddressBookInterface(_addressbook); // Stored but never used
}
```

However, throughout the entire contract, addressbook is never referenced in any function:

- setPriceTimeValidity - doesn't use it
- setDeviationMultiplier - doesn't use it
- setExpiryPriceInOracle - doesn't use it

- `getPrice` - doesn't use it
- `getHistoricalPrice` - doesn't use it

The variable exists only as a public state variable that can be read, but no internal logic depends on it.

Impact

1. **Unnecessary Storage Slot:** The `addressbook` occupies an entire storage slot (32 bytes), increasing contract storage size and deployment gas cost.
2. **Initialization Overhead:** During contract initialization, gas is wasted storing a value that will never be read or used.
3. **Code Confusion:** Developers reviewing the contract may assume `addressbook` is important to the contract's logic and waste time understanding how it's used, only to discover it serves no purpose.
4. **Upgrade Burden:** If the contract is ever upgraded, the unused state variable must be maintained for storage layout compatibility, even though it provides no functionality.

Code Snippet

```
/// @notice the enhanced addressbook address
AddressBookInterface public addressbook;
```

Tool Used

Manual Review

Recommendation

Remove the unused `addressbook` state variable entirely. This involves three changes:

1. Remove the state variable declaration:

```
// DELETE THIS:
/// @notice the enhanced addressbook address
// AddressBookInterface public addressbook;
```

2. Remove it from the initialize function parameter and initialization logic:

```
// BEFORE:
function initialize(address _bot, address _asset, address _oracle, address
↪ _addressbook, address _owner)
```

```

    public
    initializer
    {
        require(_bot != address(0), "ManualPricer: Cannot set 0 address as bot");
        require(_oracle != address(0), "ManualPricer: Cannot set 0 address as oracle");
        require(_addressbook != address(0), "ManualPricer: Cannot set 0 address as
        ↪ addressbook");
        __Ownable_init(_owner);
        bot = _bot;
        oracle = OracleInterface(_oracle);
        asset = _asset;
        addressbook = AddressBookInterface(_addressbook);
    }

    // AFTER:
    function initialize(address _bot, address _asset, address _oracle, address _owner)
    public
    initializer
    {
        require(_bot != address(0), "ManualPricer: Cannot set 0 address as bot");
        require(_oracle != address(0), "ManualPricer: Cannot set 0 address as oracle");
        __Ownable_init(_owner);
        bot = _bot;
        oracle = OracleInterface(_oracle);
        asset = _asset;
    }

```

3. Remove the unused import if AddressBookInterface is only used for addressbook:

```

// Check if AddressBookInterface is used elsewhere before removing
// Currently it's only imported for the unused addressbook variable
// import {AddressBookInterface} from "./interfaces/AddressBookInterface.sol"; //
↪ Can be removed

```

Benefits of this change:

- Saves 1 storage slot, reducing deployment gas cost
- Saves gas on initialization (no longer need to store the addressbook address)
- Reduces code complexity and confusion
- Simplifies the contract interface
- Makes the contract's dependencies clearer (it only needs Oracle, not AddressBook)

Discussion

6xiaowu9

Acknowledged. Thanks. will keep it.

Issue L-14: getPrice And getChainlinkRoundData Don't Validate Pricer Return Values To Be Non-zero [RE-SOLVED]

Source:

<https://github.com/sherlock-audit/2026-05-enhanced-may-26th-2026/issues/64>

Summary

The `getPrice` and `getChainlinkRoundData` functions in `Oracle` do not validate that pricer return values are non-zero, allowing a malfunctioning or malicious pricer to propagate zero prices silently into the system without any indication of error.

Vulnerability Detail

Both `getPrice` and `getChainlinkRoundData` call external pricers but do not validate their return values for validity:

In `getPrice`:

```
function getPrice(address _asset) external view returns (uint256) {
    uint256 price = stablePrice[_asset];

    if (price == 0) {
        require(assetPricer[_asset] != address(0), "Oracle: Pricer for this asset
        ↳ not set");
        price = EnhancedPricerInterface(assetPricer[_asset]).getPrice();
    }

    return price;
}
```

In `getChainlinkRoundData`:

```
function getChainlinkRoundData(address _asset, uint80 _roundId) external view
↳ returns (uint256, uint256) {
    uint256 price = stablePrice[_asset];
    uint256 timestamp = block.timestamp;

    if (price == 0) {
        require(assetPricer[_asset] != address(0), "Oracle: Pricer for this asset
        ↳ not set");

        (price, timestamp) = EnhancedPricerInterface(assetPricer[_asset]).getHistori
        ↳ calPrice(_roundId);
    }
}
```

```
    return (price, timestamp);  
}
```

The problem: If a pricer returns 0 as price or timestamp, this value is returned directly without any validation. There is no check to ensure:

- The price is greater than zero
- The timestamp is reasonable (not zero or in the future)
- The pricer actually succeeded in fetching a valid price

A zero price could indicate:

- Pricer malfunction or failure
- Pricer not initialized
- Data unavailable
- Intentional error signal

But to the Oracle caller, it looks like a valid price of 0 USD, which is economically nonsensical for any asset.

Impact

If a pricer malfunctions and returns 0, downstream contracts receive this as a legitimate price without indication of failure. Unlike an explicit revert, returning 0 is a silent failure that may go unnoticed until significant damage occurs. Monitoring systems won't detect the failure. Callers cannot distinguish between "price legitimately is 0" and "pricer failed to fetch a price". This violates the fail-secure principle.

Code Snippet

getPrice:

```
function getPrice(address _asset) external view returns (uint256) {  
    uint256 price = stablePrice[_asset];  
  
    if (price == 0) {  
        require(assetPricer[_asset] != address(0), "Oracle: Pricer for this asset  
        ↳ not set");  
        price = EnhancedPricerInterface(assetPricer[_asset]).getPrice();  
    }  
  
    return price;  
}
```

getChainlinkRoundData:

```

function getChainlinkRoundData(address _asset, uint80 _roundId) external view
↳ returns (uint256, uint256) {
    uint256 price = stablePrice[_asset];
    uint256 timestamp = block.timestamp;

    if (price == 0) {
        require(assetPricer[_asset] != address(0), "Oracle: Pricer for this asset
↳ not set");

        (price, timestamp) = EnhancedPricerInterface(assetPricer[_asset]).getHistoricalPrice(_roundId);
↳ icalPrice(_roundId);
    }

    return (price, timestamp);
}

```

Tool Used

Manual Review

Recommendation

Add validation to ensure pricer return values are non-zero and valid before returning them:

```

function getPrice(address _asset) external view returns (uint256) {
    uint256 price = stablePrice[_asset];

    if (price == 0) {
        require(assetPricer[_asset] != address(0), "Oracle: Pricer for this asset
↳ not set");

        price = EnhancedPricerInterface(assetPricer[_asset]).getPrice();

        // ADD THIS VALIDATION
        require(price > 0, "Oracle: Pricer returned invalid price (0)");
    }

    return price;
}

```

```

function getChainlinkRoundData(address _asset, uint80 _roundId) external view
↳ returns (uint256, uint256) {
    uint256 price = stablePrice[_asset];
    uint256 timestamp = block.timestamp;

```

```

    if (price == 0) {
        require(assetPricer[_asset] != address(0), "Oracle: Pricer for this asset
        ↪ not set");

        (price, timestamp) = EnhancedPricerInterface(assetPricer[_asset]).getHistoricalPrice(_roundId);

        // ADD THIS VALIDATION
        require(price > 0, "Oracle: Pricer returned invalid price (0)");
        require(timestamp > 0 && timestamp <= block.timestamp, "Oracle: Pricer
        ↪ returned invalid timestamp");
    }

    return (price, timestamp);
}

```

Additional Considerations:

1. **Upper Bounds Check:** Consider adding maximum price bounds to detect pricer overflow or manipulation:

```

require(price <= MAX_PRICE, "Oracle: Price exceeds maximum allowed value");

```

2. **Timestamp Sanity:** Ensure historical prices have timestamps that are:

- Greater than the asset's deployment time
- Less than or equal to current block.timestamp
- Not recently manipulated (e.g., within last N seconds)

3. **Pricer Health Monitoring:** Track failed pricer calls and implement fallback mechanisms:

```

uint256 failedPricerCalls[address]; // Track failures per pricer
// After N consecutive failures, disable pricer or require manual intervention

```

4. **Event on Validation Failure:** Emit an event when a pricer returns invalid data for monitoring:

```

event InvalidPricerReturnValue(address indexed asset, address indexed pricer,
    ↪ uint256 returnedValue);

```

5. **Consider Reverting vs Returning:** Decide if returning 0 should revert (fail-secure) or allow callers to handle it. Reverting is safer:

```

// Fail-secure approach (recommended)
require(price > 0, "Oracle: unable to get valid price");

```

This ensures that pricer failures are caught immediately rather than propagating invalid data through the system.

Discussion

6xiaowu9

Fixed. commit ff49e21, 1442088; fix: validate historical oracle rounds

Confirmed. The live-price portion had already been fixed by rejecting zero values returned from `getPrice`. We have now also fixed the remaining historical-round path. For non-stable assets, `getChainlinkRoundData` validates that the returned historical price is non-zero, the timestamp is non-zero, and the timestamp is not in the future. Invalid pricer responses now revert instead of being silently propagated. The stable-price path remains unchanged and returns the configured stable price with the current block timestamp. Regression tests cover zero historical prices, zero timestamps, future timestamps, valid historical rounds, and stable-price behavior.

Issue L-15: Asset Pricer Being Maintained After Settlement Is Not Enforced On Chain [ACKNOWLEDGED]

Source:

<https://github.com/sherlock-audit/2026-05-enhanced-may-26th-2026/issues/65>

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

The `Oracle` relies on an operational assumption that asset pricers remain unchanged between price submission and settlement completion, but this constraint is not technically enforced in the contract, creating risk if pricer assignments are inadvertently modified.

Vulnerability Detail

The `isLockingPeriodOver` and `isDisputePeriodOver` functions look up the currently assigned pricer to determine validity windows:

```
function isLockingPeriodOver(address _asset, uint256 _expiryTimestamp) public view
↳ returns (bool) {
    uint256 price = stablePrice[_asset];

    if (price == 0) {
        address pricer = assetPricer[_asset]; // Looks up CURRENT pricer
        uint256 lockingPeriod = pricerLockingPeriod[pricer];

        return block.timestamp > _expiryTimestamp + lockingPeriod;
    }

    return true;
}

function isDisputePeriodOver(address _asset, uint256 _expiryTimestamp) public view
↳ returns (bool) {
    uint256 price = stablePrice[_asset];

    if (price == 0) {
        Price memory storedPriceData = storedPrice[_asset][_expiryTimestamp];
        if (storedPriceData.timestamp == 0) {
            return false;
        }

        address pricer = assetPricer[_asset]; // Looks up CURRENT pricer
        uint256 disputePeriod = pricerDisputePeriod[pricer];
```

```

        return block.timestamp > storedPriceData.timestamp + disputePeriod;
    }

    return true;
}

```

However, the `setAssetPricer` function allows changing a pricer at any time with no restrictions:

```

function setAssetPricer(address _asset, address _pricer) external onlyOwner {
    require(_pricer != address(0), "Oracle: cannot set pricer to address(0)");
    require(stablePrice[_asset] == 0, "Oracle: could not set a pricer for stable
    ↳ asset");

    assetPricer[_asset] = _pricer; // Can be changed anytime, no guards

    emit PricerUpdated(_asset, _pricer);
}

```

During the audit it was indicated that "The asset pricer should be maintained after settlement, not before settlement." However, this is a **governance discipline requirement**, not a technical enforcement. An owner could accidentally or intentionally call `setAssetPricer` while prices are in their locking or dispute periods, causing:

1. Wrong locking/dispute periods retroactively applied to existing prices
2. Prices settling with unintended parameters
3. Dispute windows unexpectedly shortened or extended mid-dispute

Example scenario:

- Time 1000: ManualPricer A set for BTC with 3600s dispute period
- Time 1010: Bot submits BTC price at timestamp 1000
- Time 1020: Owner accidentally calls `setAssetPricer(BTC, ManualPricer B)` (with 300s dispute period)
- Time 1330: `isDisputePeriodOver(BTC, 1000)` checks with Pricer B
 - Returns: $1330 > 1010 + 300 \rightarrow \text{TRUE}$
 - Price finalized early with wrong dispute period!

Impact

1. **Retroactive Parameter Changes:** Pricer changes unexpectedly affect in-flight price disputes, potentially finalizing prices too early.
2. **Dispute Window Manipulation:** An owner or attacker could shorten dispute periods mid-dispute to finalize before disputer can react.

3. **Silent Finalization:** Price dispute periods could be cut short without any indication that parameters changed, making auditing difficult.
4. **Settlement Inconsistency:** Prices may settle using different parameters than were active when submitted, violating the expected economic model.

Code Snippet

isLockingPeriodOver:

```
function isLockingPeriodOver(address _asset, uint256 _expiryTimestamp) public view
↳ returns (bool) {
    uint256 price = stablePrice[_asset];

    if (price == 0) {
        address pricer = assetPricer[_asset]; // Looks up CURRENT pricer
        uint256 lockingPeriod = pricerLockingPeriod[pricer];

        return block.timestamp > _expiryTimestamp + lockingPeriod;
    }

    return true;
}
```

isDisputePeriodOver:

```
function isDisputePeriodOver(address _asset, uint256 _expiryTimestamp) public view
↳ returns (bool) {
    uint256 price = stablePrice[_asset];

    if (price == 0) {
        Price memory storedPriceData = storedPrice[_asset][_expiryTimestamp];
        if (storedPriceData.timestamp == 0) {
            return false;
        }

        address pricer = assetPricer[_asset]; // Looks up CURRENT pricer
        uint256 disputePeriod = pricerDisputePeriod[pricer];

        return block.timestamp > storedPriceData.timestamp + disputePeriod;
    }

    return true;
}
```

setAssetPricer:

```
function setAssetPricer(address _asset, address _pricer) external onlyOwner {
    require(_pricer != address(0), "Oracle: cannot set pricer to address(0)");
}
```

```

    require(stablePrice[_asset] == 0, "Oracle: could not set a pricer for stable
    ↪  asset");

    assetPricer[_asset] = _pricer; // Can be changed anytime, no guards

    emit PricerUpdated(_asset, _pricer);
}

```

setAssetPricer (lines 95-105) - no temporal guards

isLockingPeriodOver (lines 328-340) - uses current pricer

isDisputePeriodOver (lines 347-365) - uses current pricer

setExpiryPrice (lines 167-173) - stores price without recording pricer used

Tool Used

Manual Review

Recommendation

Technically enforce the operational constraint by storing the pricer used at the time of price submission:

```

/// @dev mapping between asset, expiry timestamp, and the Price structure at the
↪  expiry timestamp
mapping(address => mapping(uint256 => Price)) internal storedPrice;

/// @dev mapping between asset, expiry timestamp, and the pricer that submitted the
↪  price
mapping(address => mapping(uint256 => address)) internal pricerUsedForPrice;

/**
 * @notice submits the expiry price to the oracle, can only be set from the pricer
 * @dev asset price can only be set after the locking period is over and before the
↪  dispute period has started
 * @param _asset asset address
 * @param _expiryTimestamp expiry timestamp
 * @param _price asset price at expiry
 */
function setExpiryPrice(address _asset, uint256 _expiryTimestamp, uint256 _price)
↪  external {
    require(msg.sender == assetPricer[_asset], "Oracle: caller is not authorized to
    ↪  set expiry price");
    require(isLockingPeriodOver(_asset, _expiryTimestamp), "Oracle: locking period
    ↪  is not over yet");
    require(storedPrice[_asset][_expiryTimestamp].timestamp == 0, "Oracle: dispute
    ↪  period started");
}

```

```

    storedPrice[_asset][_expiryTimestamp] = Price({price: _price, timestamp:
    ↪ block.timestamp});
    pricerUsedForPrice[_asset][_expiryTimestamp] = msg.sender; // Record which
    ↪ pricer submitted
    emit ExpiryPriceUpdated(_asset, _expiryTimestamp, _price, block.timestamp);
}

/**
 * @notice check if the locking period is over for setting the asset price at a
 * ↪ particular expiry timestamp
 * @param _asset asset address
 * @param _expiryTimestamp expiry timestamp
 * @return True if locking period is over, False if not
 */
function isLockingPeriodOver(address _asset, uint256 _expiryTimestamp) public view
    ↪ returns (bool) {
    uint256 price = stablePrice[_asset];

    if (price == 0) {
        // Use the pricer that was active when price was submitted
        address pricer = pricerUsedForPrice[_asset][_expiryTimestamp];
        if (pricer == address(0)) {
            // Price not submitted yet, check current pricer
            pricer = assetPricer[_asset];
        }
        uint256 lockingPeriod = pricerLockingPeriod[pricer];

        return block.timestamp > _expiryTimestamp + lockingPeriod;
    }

    return true;
}

/**
 * @notice check if the dispute period is over
 * @param _asset asset address
 * @param _expiryTimestamp expiry timestamp
 * @return True if dispute period is over, False if not
 */
function isDisputePeriodOver(address _asset, uint256 _expiryTimestamp) public view
    ↪ returns (bool) {
    uint256 price = stablePrice[_asset];

    if (price == 0) {
        Price memory storedPriceData = storedPrice[_asset][_expiryTimestamp];
        if (storedPriceData.timestamp == 0) {
            return false;
        }
    }
}

```

```

        // Use the pricer that was active when price was submitted
        address pricer = pricerUsedForPrice[_asset][_expiryTimestamp];
        uint256 disputePeriod = pricerDisputePeriod[pricer];

        return block.timestamp > storedPriceData.timestamp + disputePeriod;
    }

    return true;
}

```

This ensures:

- Each price is permanently associated with the pricer and parameters used at submission time
- Pricer changes after submission do not retroactively affect locking/dispute periods
- Dispute windows cannot be shortened mid-flight
- Full audit trail of which pricer was used for which price

Alternatively, add preventive checks to `setAssetPricer` to prevent updates during active disputes:

```

function setAssetPricer(address _asset, address _pricer) external onlyOwner {
    require(_pricer != address(0), "Oracle: cannot set pricer to address(0)");
    require(stablePrice[_asset] == 0, "Oracle: could not set a pricer for stable
    ↪ asset");

    // ADD: Prevent pricer changes during disputes (simplified check)
    require(block.timestamp > lastDisputeTime[_asset] + SAFE_UPDATE_WINDOW,
        "Oracle: cannot update pricer while disputes are active");

    assetPricer[_asset] = _pricer;
    emit PricerUpdated(_asset, _pricer);
}

```

The first approach (storing historical pricers) is more robust as it guarantees correctness regardless of when changes occur.

Discussion

6xiaowu9

We consider this issue invalid as a security vulnerability. Both `setAssetPricer` and `setDisputePeriod` are restricted to the trusted Oracle owner. The owner is also the UUPS upgrade authority and can replace the Oracle implementation itself, so changing the effective dispute policy does not grant this role any capability it does not already possess. The current behavior intentionally applies the configuration associated with the currently active pricer. Changing a pricer is a governance/configuration operation and

must include selecting the intended locking and dispute periods. An accidental change during an active dispute would therefore be an administrative misconfiguration, not an unprivileged attack path. No external user can change the pricer, shorten the dispute period, force early finalization, or otherwise reach the reported state. Snapshotting a per-expiry dispute deadline could provide defense in depth against operational mistakes, but it is not required to enforce a security boundary under the current trust model.

cwkang1998

We acknowledge this as a general health issue.

Issue L-16: Default liquidationMultiplier is lost behind the proxy [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-05-enhanced-may-26th-2026/issues/67>

Summary

liquidationMultiplier is given a default value of 10_000 (100%) via an inline state-variable initializer. MarginCalculator is deployed behind a UUPS proxy, so this assignment executes in the implementation contract's constructor context and writes to the implementation's storage – never to the proxy's storage. On the live proxy the variable remains 0 until the owner explicitly calls setLiquidationMultiplier.

Vulnerability Detail

Inline initializers are equivalent to constructor assignments. For an upgradeable contract, all state that must live in the proxy has to be set inside initialize(), because the constructor runs only against the implementation contract and the proxy's storage is what is actually read at runtime.

initialize() does not set liquidationMultiplier:

```
function initialize(address _oracle, address _addressBook, address _owner) external  
↳ initializer {  
    require(_oracle != address(0), "MarginCalculator: invalid oracle address");  
    __Ownable_init(_owner);  
    oracle = OracleInterface(_oracle);  
    addressBook = AddressBookInterface(_addressBook);  
    ZERO = FPI.fromScaledUint(0, BASE);  
}
```

So through the proxy liquidationMultiplier == 0. It is consumed in _getDebtPrice:

```
return price.toScaledUint(_collateralDecimals, true) * liquidationMultiplier /  
↳ MAX_BPS;
```

With the multiplier at 0, the debt price returned by _getDebtPrice (and therefore the debt Price returned to the liquidation flow) is 0 instead of the intended 100%-of-price value.

Impact

Until the owner manually calls setLiquidationMultiplier(10_000), the documented 100% default never takes effect on the deployed proxy. Liquidation debt price computes

to 0, so the liquidation pricing path is broken (returns zero collateral per repaid oToken). Additionally, type 1 vaults (liquidatable) are not supported with the current configuration.

Code Snippet

<https://github.com/sherlock-audit/2026-05-enhanced-may-26th-2026/blob/b2e0082f695d5f0a82d8a5051d69423ad4a7cba4/contracts/src/core/MarginCalculator.sol#L91>

<https://github.com/sherlock-audit/2026-05-enhanced-may-26th-2026/blob/b2e0082f695d5f0a82d8a5051d69423ad4a7cba4/contracts/src/core/MarginCalculator.sol#L1009>

Tool Used

Manual Review

Recommendation

Set the default inside `initialize()` instead of relying on the inline initializer:

```
function initialize(address _oracle, address _addressBook, address _owner) external  
↳ initializer {  
    ...  
    liquidationMultiplier = 10_000;  
}
```

Discussion

6xiaowu9

Fixed. commit 53fee2e (fix: init liquidationMultiplier)

Issue L-17: Marginability equality check is overridden by the covered whitelist for type 0/2 vaults [ACKNOWLEDGED]

Source:

<https://github.com/sherlock-audit/2026-05-enhanced-may-26th-2026/issues/68>

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

In `_isMarginableCollateral`, the check requiring the vault's deposited collateral to equal the short oToken's declared collateral asset is discarded for vault types 0 and 2 and replaced by a whitelist-membership check (`isCoveredWhitelistedCollateral`). The replacement does not compare the deposited asset against the oToken's collateral, so in principle a vault backed by asset A could secure an oToken declaring collateral asset B when both A and B are whitelisted covered collateral for the underlying. In the current deployment configuration this is not reachable, so the issue is a latent / defense-in-depth weakness rather than an exploitable bug.

Vulnerability Detail

The function computes the correct constraint and then unconditionally overwrites it for type 0/2 vaults:

```
if (_vaultDetails.hasShort) {
    isMarginable = _vaultDetails.shortCollateralAsset == _vault.collateralAssets[0];
    if (_vaultDetails.vaultType == 0 || _vaultDetails.vaultType == 2) {
        WhitelistInterface whitelist =
            ↪ WhitelistInterface(addressBook.getWhitelist());
        isMarginable = whitelist.isCoveredWhitelistedCollateral(
            ↪ _vault.collateralAssets[0], _vaultDetails.shortUnderlyingAsset,
            ↪ _vaultDetails.isShortPut
        );
    }
}
```

`isCoveredWhitelistedCollateral` validates only (collateral, underlying, isPut) against the whitelist; it never compares the deposited collateral to the oToken's `shortCollateralAsset`. The earlier equality result is thrown away, so the equality is no longer enforced for type 0/2 vaults.

Notes on current exploitability:

1. Through `EnhancedOptions`: vaults are created in `_executeNewPosition`, which deploys/fetches the oToken with `collateral: sellerConfirmation.collateralAsset`

and deposits `sellerConfirmation.collateralAsset`, the same field. The deposited asset and the oToken's declared collateral are therefore always identical, so a mismatch can never be constructed via this contract.

2. Through a direct `Controller.operate` call: every vault-mutating action is gated by `onlyAuthorized(msg.sender, _args.owner)`, which (via `_isAuthorized`) requires `msg.sender == manager` whenever `operatorsEnabled == false`. `operatorsEnabled` defaults to `false`, and the manager is the `EnhancedOptions` router. So arbitrary users cannot open a self-owned vault with mismatched assets unless the owner first calls `setOperatorsEnabled(true)`.

Reaching the mismatch therefore requires the admin to both (a) whitelist two or more distinct covered collateral assets for the same underlying and (b) enable direct user operation (`operatorsEnabled = true`). Neither is the case in the intended configuration.

Impact

No impact in the current/intended deployment: the only vault-creation entry point (`EnhancedOptions`) cannot produce a collateral mismatch, and direct `Controller` access is restricted to the `EnhancedOptions` manager while `operatorsEnabled` is `false`. The override is a latent correctness gap. If direct user operation is ever enabled alongside multiple whitelisted covered collaterals for one underlying, a writer could back a short with one asset while the oToken settles in another, breaking the 1:1 collateral-to-settlement correspondence and exposing the shared `MarginPool` to under-collateralization. As deployed, this is defense-in-depth only.

Code Snippet

<https://github.com/sherlock-audit/2026-05-enhanced-may-26th-2026/blob/b2e0082f695d5f0a82d8a5051d69423ad4a7cba4/contracts/src/core/MarginCalculator.sol#L1186-L1195>

Tool Used

Manual Review

Recommendation

Treat the whitelist check as an additional constraint rather than a replacement, so the deposited collateral must still equal the oToken's settlement asset:

```
if (_vaultDetails.hasShort) {
    isMarginable = _vaultDetails.shortCollateralAsset == _vault.collateralAssets[0];
    if (_vaultDetails.vaultType == 0 || _vaultDetails.vaultType == 2) {
        WhitelistInterface whitelist =
            ↳ WhitelistInterface(addressBook.getWhitelist());
```

```
        isMarginable = isMarginable && whitelist.isCoveredWhitelistedCollateral(  
            _vault.collateralAssets[0], _vaultDetails.shortUnderlyingAsset,  
            ↪ _vaultDetails.isShortPut  
        );  
    }  
}
```

Discussion

6xiaowu9

`_isMarginableCollateral` does indeed use the covered-whitelist result for type 0/2 and overrides the earlier `shortCollateralAsset == vault.collateral` check.

However, in the complete entry flow, `EnhancedOptions` uses the same `sellerConfirmation.collateralAsset` both to create the `oToken` and to perform `DepositCollateral`. In addition, the `Controller` only allows calls from the manager, so regular users cannot manually combine mismatched assets.

Therefore, the current system cannot construct the collateral mismatch required by the audit report. This is a defensive correctness gap, not an existing reachable vulnerability.

cwkang1998

We acknowledge this as a low/informational issue.

Issue L-18: Redundant operator authorization machinery in the controller [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-05-enhanced-may-26th-2026/issues/69>

Summary

The controller carries a full account-operator authorization system (operators mapping, setOperator, isOperator, setOperatorsEnabled, the operator branches in `_isAuthorized`, and the operatorsEnabled conditional in `_revertIfNotManager`) even though, by design, only the manager (the EnhancedOptions router) is meant to act on vaults. `operatorsEnabled` is initialized to `false` and is not intended to be enabled. The operator paths are therefore dead code, and they also introduce a latent authorization bypass if `operatorsEnabled` were ever set to `true`.

Vulnerability Detail

Vault-mutating actions are guarded by `onlyAuthorized`, which calls `_isAuthorized`:

```
function _isAuthorized(address _sender, address _accountOwner) internal view {
    require(
        (_sender == manager) || (operators[_accountOwner][_sender] &&
        → operatorsEnabled)
        || (_sender == _accountOwner && operatorsEnabled),
        "C6"
    );
}
```

While `operatorsEnabled == false` (the default), only `msg.sender == manager` can pass, so the operator and self-owner branches are inert. If `operatorsEnabled` is flipped to `true`, those branches activate and any vault owner (or an approved operator) can call vault actions directly, bypassing the EnhancedOptions router. That router invariant is what keeps deposited collateral aligned with the `oToken`'s declared collateral, so enabling operators would also reopen the collateral-mismatch surface in `_isMarginableCollateral`.

The companion helper has the same shape:

```
function _revertIfNotManager() internal view {
    if (!operatorsEnabled) {
        require(msg.sender == manager, "C6");
    }
}
```

When `operatorsEnabled == true` the manager check is skipped entirely, leaving the entrypoints guarded by `_revertIfNotManager` callable by anyone.

Since the manager is the sole intended caller, the entire operator subsystem (`operatorsEnabled`, the operators mapping, `setOperator`, `isOperator`, `setOperatorsEnabled`, the `AccountOperatorUpdated` event, and the conditional branches above) is unnecessary.

Impact

No impact in the default configuration, where `operatorsEnabled == false` and only the manager can operate. The issue is dead/over-complex authorization code plus a latent risk: if the owner ever enables operators, `_isAuthorized` and `_revertIfNotManager` would permit non-manager callers, breaking the single-entry-point assumption the rest of the system relies on.

Code Snippet

<https://github.com/sherlock-audit/2026-05-enhanced-may-26th-2026/blob/b2e0082f695d5f0a82d8a5051d69423ad4a7cba4/contracts/src/core/Controller.sol#L226-L232>

<https://github.com/sherlock-audit/2026-05-enhanced-may-26th-2026/blob/b2e0082f695d5f0a82d8a5051d69423ad4a7cba4/contracts/src/core/Controller.sol#L234-L239>

Tool Used

Manual Review

Recommendation

Simplify the authorization to always require the manager, and remove the now-unused operator state and functions:

```
function _isAuthorized(address _sender, address /*_accountOwner*/) internal view {
    require(_sender == manager, "C6");
}

function _revertIfNotManager() internal view {
    require(msg.sender == manager, "C6");
}
```

Discussion

6xiaowu9

The potential authorization bypass has already been addressed by 5355a6d, 30e8919, and 33f8956. What remains is only a dead compatibility interface.

Issue L-19: Rounding mismatch in redemption-balance excess-removal guard for low-decimal collateral [ACKNOWLEDGED]

Source:

<https://github.com/sherlock-audit/2026-05-enhanced-may-26th-2026/issues/70>

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

`_removeExcessCollateralFromRedemptionBalances()` decides whether the ITM-call excess has already been trimmed by comparing the stored `collateralRedemptionBalance` against `oTokenQuantity` re-scaled to collateral decimals. Both the per-mint increment and this comparison floor-divide by $1e8$. When the collateral has fewer than 8 decimals, the per-mint floors can accumulate a different total than the single floor applied to the summed quantity, so the equality guard can fail and the excess-removal path is skipped. This rounding gap is real arithmetically but, as the developer notes, is not reachable in the current product configuration.

Vulnerability Detail

On each physically settled call mint, the redemption collateral balance is incremented with a per-mint floor:

```
int256(_args.amount * (10 **  
    ↪ uint256(IERC20Metadata(oToken.collateralAsset()).decimals())) / 1e8)
```

At settlement/redeem, the guard recomputes the expected balance from the aggregate `oToken` quantity with a single floor:

```
uint256 oTokenQuantityCollateralDecimals =  
    oTokenQuantity * (10 **  
    ↪ uint256(IERC20Metadata(oToken.collateralAsset()).decimals())) / 1e8;  
  
if (collateralRedemptionBalance != oTokenQuantityCollateralDecimals) return;
```

$\text{sum}(\text{floor}(\text{amount}_i * \text{scale} / 1e8))$ is not always equal to $\text{floor}(\text{sum}(\text{amount}_i) * \text{scale} / 1e8)$ when $\text{scale} < 1e8$ (collateral decimals < 8). With a 6-decimal collateral:

```
mint A: amount = 100000050 -> += floor(100000050 * 1e6 / 1e8) = floor(1000000.5) =  
    ↪ 1000000  
mint B: amount = 100000050 -> += 1000000  
collateralRedemptionBalance = 2000000
```

```
otokenQuantity = 200000100
otokenQuantityCollateralDecimals = floor(200000100 * 1e6 / 1e8) = 2000001

2000000 != 2000001 -> guard returns early, excess removal skipped
```

The guard is meant to be true only after excess has already been removed; here it reads as already-removed when it has not been, so the ITM-call excess trimming never runs.

Notes on the current configuration and exploitability:

Physically settled positions are opened by EnhancedOptions as vault type 2, and must pass `Controller._verifyFinalState()` and `MarginCalculator._isMarginableCollateral()`. For calls, `Whitelist.whitelistCoveredCollateral(collateral, underlying, false)` requires `collateral == underlying`, otherwise it reverts with "Whitelist: covered call collateral must be underlying". So a type 2 call vault with 18 decimals underlying and a separate 6-decimal LST collateral cannot exist.

If underlying and collateral are the same 6-decimal asset, their expiry prices are equal, so the appreciation-based excess computed by `_removeExcessCollateralFromRedemptionBalances()` ($1 - \text{underlyingExpiryPrice} / \text{collateralExpiryPrice}$) is zero. Even when the rounding makes the equality guard return early, there is no appreciation surplus to remove, so no funds are stranded.

This was confirmed against the code: `whitelistCoveredCollateral` enforces `collateral == underlying` for calls, and type 2 call vaults are constrained to covered collateral through `_isMarginableCollateral`.

Impact

No impact under the current configuration. Because covered calls force `collateral == underlying`, the excess that the skipped branch would remove is always zero, so the early return is harmless and no collateral is stranded.

Code Snippet

<https://github.com/sherlock-audit/2026-05-enhanced-may-26th-2026/blob/b2e0082f695d5f0a82d8a5051d69423ad4a7cba4/contracts/src/core/ControllerLogic.sol#L270-L272>

<https://github.com/sherlock-audit/2026-05-enhanced-may-26th-2026/blob/b2e0082f695d5f0a82d8a5051d69423ad4a7cba4/contracts/src/core/ControllerLogic.sol#L589-L592>

<https://github.com/sherlock-audit/2026-05-enhanced-may-26th-2026/blob/b2e0082f695d5f0a82d8a5051d69423ad4a7cba4/contracts/src/core/Whitelist.sol#L213-L214>

Tool Used

Manual Review

Recommendation

If low-decimal call collateral that differs from the underlying is ever to be supported, do not rely on exact equality between a sum of per-mint floors and a single floor of the aggregate. As of now, no change is required because the covered-call collateral == underlying constraint makes the excess zero.

Discussion

6xiaowu9

Sequential minting with flooring may differ from flooring after aggregation, so the mathematical observation is valid.

However, a physical call must go through `Whitelist.whitelistCoveredCollateral`, and this entry point enforces that the collateral for a call must be equal to the underlying. Since the two are the same, the appreciation excess to be removed is zero. Even if the exact-value guard returns early due to rounding, there is no cross-asset excess that could be left behind.

Regular users also cannot bypass the manager and the final-state whitelist check to create a type-2 call with different LST/underlying assets. Therefore, there is currently no asset-retention impact.

cwkang1998

As mentioned, this is not part of the supported product flow. We acknowledge this as informational.

Issue L-20: Naked pool balance is not reduced by collateral paid out to option holders [ACKNOWLEDGED]

Source:

<https://github.com/sherlock-audit/2026-05-enhanced-may-26th-2026/issues/71>

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

`nakedPoolBalance[asset]` is the meter that caps total collateral locked in naked (type 1) margin vaults via `nakedCap`. It is increased by the full deposit when a naked vault is funded, but at settlement it is only reduced by the writer's returned excess (`collateralPayout`). The portion of the writer's collateral that is paid out to option holders on redeem is never subtracted. As a result the meter monotonically over-counts after each in-the-money naked settlement, drifting upward relative to the collateral actually held.

Vulnerability Detail

On deposit into a type 1 vault, the meter is increased by the entire deposited amount and checked against the cap:

```
if (typeVault == 1) {
    controller.increaseNakedPoolBalance(_args.asset, _args.amount);
    require(controller.getNakedPoolBalance(_args.asset) <=
        ↪ controller.getNakedCap(_args.asset), "C37");
}
```

When holders redeem an ITM cash-settled option, their payout is transferred out of the pool's collateral, but `nakedPoolBalance` is not touched (redeem is otoken-level and not type-aware):

```
uint256 payout = getPayout(_args.otoken, _args.amount); // Dn{col}
otoken.burnOtoken(sender, _args.amount);
pool.transferToUser(collateral, _args.receiver, payout);
```

At settlement of the naked vault, the meter is reduced only by what is returned to the writer:

```
if (typeVault == 1) {
    // @audit paid to option holders is not reduced
    controller.reduceNakedPoolBalance(settleMem.collateral,
        ↪ settleMem.collateralPayout);
}
```

For an ITM naked position, `collateralPayout` (the writer's excess) is strictly less than the collateral originally deposited, because part of that collateral was already paid to holders via redeem. The difference (the holders' share) is added on deposit but never removed, so `nakedPoolBalance` retains a permanent residual for every ITM naked settlement. The withdraw path (`reduceNakedPoolBalance(_args.asset, _args.amount)`) only accounts for the writer's own withdrawals and does not close this gap.

Impact

No fund loss: the actual token transfers (pool to holders, pool to writer) are correct. The defect is confined to the `nakedPoolBalance` accounting meter, which ratchets upward over time. The consequence is that `nakedCap` is consumed by phantom balance, so new naked-margin deposits for that collateral asset start reverting with "C37" earlier than they should, and eventually the naked-margin feature for that asset becomes unusable even though real exposure is below the cap. This is a liveness/DoS drift on the naked-margin feature, mitigable by the owner raising `nakedCap`, not a loss of user funds.

Reachability in the current configuration: type 1 (naked) vaults are not created through the live entry flow. `EnhancedOptions._executeNewPosition` opens vaults only as type `isPhysicallySettled ? 2 : 0`, never type 1, and direct `Controller.operate` calls are restricted to the manager (`EnhancedOptions`) while `operatorsEnabled == false`. So the drift is latent today and only becomes relevant if type 1 vaults are ever enabled.

Code Snippet

<https://github.com/sherlock-audit/2026-05-enhanced-may-26th-2026/blob/b2e0082f695d5f0a82d8a5051d69423ad4a7cba4/contracts/src/core/ControllerLogic.sol#L206-L209>

<https://github.com/sherlock-audit/2026-05-enhanced-may-26th-2026/blob/b2e0082f695d5f0a82d8a5051d69423ad4a7cba4/contracts/src/core/ControllerLogic.sol#L381-L388>

<https://github.com/sherlock-audit/2026-05-enhanced-may-26th-2026/blob/b2e0082f695d5f0a82d8a5051d69423ad4a7cba4/contracts/src/core/ControllerLogic.sol#L535-L537>

Tool Used

Manual Review

Recommendation

Reduce `nakedPoolBalance` by the collateral that leaves the naked pool on holder redemption, so the meter reflects only collateral still backing live naked positions. Because `nakedPoolBalance` is tracked per collateral asset while redeem is per `oToken` (and

cash-settled oTokens can be backed by both fully collateralized type 0 and naked type 1 vaults), the reduction cannot be applied unconditionally on every redeem. Either:

- Track naked exposure at a granularity that lets redeem attribute the payout to naked-backed collateral (for example a separate per-oToken naked-backed accumulator), and decrement `nakedPoolBalance` by that share on redeem; or
- Decrement `nakedPoolBalance` at settlement by the writer's full original deposited collateral rather than only the returned `collateralPayout`, so deposit and settlement are symmetric for the naked meter.

Discussion

6xiaowu9

In the local logic, a type-1 deposit increases the full `nakedPoolBalance`, while settlement only decreases the excess returned to the writer, which indeed leaves a residual accounting balance corresponding to the holder payout.

However, `EnhancedOptions` only creates type 0/2 vaults, and the `Controller` only accepts calls from the manager, so there is currently no public or supported entry point to create type 1.

Therefore, this DoS drift is unreachable in the latest contracts and is not an issue that existing users can trigger.

cwkang1998

We acknowledge this as informational.

Issue L-21: Overcollateralized collateral can be stranded on a physical call when collateral and underlying differ in value [ACKNOWLEDGED]

Source:

<https://github.com/sherlock-audit/2026-05-enhanced-may-26th-2026/issues/72>

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

For physically settled ITM call settlement, the per-oToken payout to buyers is computed by `getPayout` (which caps the collateral handed out so the pool stays solvent under a collateral depeg), while the writer's overcollateralization is returned via `getExcessCollateral` (which prices the collateral at its actual expiry rate). When the collateral asset and the underlying asset are different and their relative price has moved, these two valuations disagree, and a sliver of collateral can be left in the pool that is paid to neither the buyers nor the writer. This cannot actually occur under the current configuration for a valid physical call, because covered-call collateral is constrained to equal the underlying.

Vulnerability Detail

Buyer payout on redeem uses the capped payout rate:

```
function getPayout(address _otoken, uint256 _amount) public view returns (uint256) {  
    return calculator.getExpiredPayoutRate(_otoken) * _amount / (10 ** BASE);  
}
```

Writer settlement uses `getExcessCollateral`, which subtracts the margin required (priced at the actual collateral expiry rate) from the deposited collateral and can even go negative when the collateral has depegged. In settlement scenario 3 the writer then receives the strike asset plus the excess collateral:

```
} else {  
    // this is scenario 3  
    pool.updateRedemptionBalance(  
        address(otoken), -int256(settleMem.shortAmount), true,  
        ↪ -int256(settleMem.strikePayout)  
    );  
}  
...  
pool.transferToUser(settleMem.collateral, _args.to, settleMem.collateralPayout);
```

Example (ETH call, strike 2000, wstETH collateral; ETH = 2500 and wstETH = 2400 at expiry; writer deposited 12 wstETH for 10 oTokens, 2 wstETH overcollateralized):

- On redeem, each exercised oToken pays the buyer 1 wstETH (capped by `getPayout` due to the depeg) plus the strike asset, leaving 2 wstETH in the pool after all 10 are exercised.
- On settle, `getExcessCollateral` returns collateral minus required = $12 - 10.4167 = 1.583$ wstETH, paid to the writer along with the strike payout.
- Net: buyers took 10 wstETH (capped), writer took 1.583 wstETH, and 0.417 wstETH stays in the pool, owned by no one and not recoverable through the normal settle/redeem paths.

The scenario requires a physical call whose collateral asset differs from its underlying asset (wstETH vs ETH). For call options, type 0 / type 2 collateral must pass `Whitelist.whitelistCoveredCollateral(collateral, underlying, false)`, which enforces `collateral == underlying`.

Impact

No impact in the current/intended configuration: covered calls force `collateral == underlying`, so there is no collateral-vs-underlying rate divergence and nothing is stranded. The issue is latent and would only surface if a physical call were ever allowed to use a collateral asset distinct from its underlying, which the whitelist currently forbids.

Code Snippet

<https://github.com/sherlock-audit/2026-05-enhanced-may-26th-2026/blob/b2e0082f695d5f0a82d8a5051d69423ad4a7cba4/contracts/src/core/ControllerLogic.sol#L151-L159>

<https://github.com/sherlock-audit/2026-05-enhanced-may-26th-2026/blob/b2e0082f695d5f0a82d8a5051d69423ad4a7cba4/contracts/src/core/MarginCalculator.sol#L558>

<https://github.com/sherlock-audit/2026-05-enhanced-may-26th-2026/blob/b2e0082f695d5f0a82d8a5051d69423ad4a7cba4/contracts/src/core/ControllerLogic.sol#L525-L540>

Tool Used

Manual Review

Recommendation

No change is required as of now, because the covered-call `collateral == underlying` constraint eliminates the cross-asset divergence. If the protocol ever intends to support

physical calls collateralized by an asset that can depeg against the underlying, reconcile the two valuation paths so deposited collateral always nets exactly to buyer payouts plus writer excess: either cap the writer's excess using the same solvency-adjusted rate that `getPayout` applies, or sweep any residual collateral remaining after the redeem window to the writer (or a defined recipient) so no dust is stranded.

Discussion

6xiaowu9

This issue does not exist.

The report assumes that the collateral and underlying of a physical call have different values, such as `wstETH/ETH`.

In practice, `whitelistCoveredCollateral` enforces `_collateral == _underlying` for calls. The final-state check in `MarginCalculator` also requires the covered whitelist, and `EnhancedOptions` uses the same `collateral` field for both creation and deposit.

cwkang1998

As mentioned, this scenario cannot happen for the supported product feature set. Under the current supported flow, type-2 call vaults are subjected to covered-collateral validation which requires `collateral == underlying`.

Therefore, we acknowledge this as informational.

Issue L-22: Option type is inferred from whitelist membership, which can be ambiguous if a collateral is whitelisted as both covered and naked [ACKNOWLEDGED]

Source:

<https://github.com/sherlock-audit/2026-05-enhanced-may-26th-2026/issues/73>

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

`MarginCalculator.getOptionType()` decides whether a position uses the covered or naked margin formula from collateral-whitelist membership alone, checking the covered whitelist first. The covered and naked whitelists are independent and not mutually exclusive, so the same `(collateral, underlying, isPut)` tuple can be whitelisted as both, in which case the covered branch always wins. This is an inherited behavior from the core margin system. It is not reachable in the audited configuration, where only the covered-call path is configured and naked-margin (type 1) vaults are not created through any live entry point, so it is an informational design observation rather than an exploitable issue.

Vulnerability Detail

`getOptionType` resolves the type from two whitelist reads, with covered taking priority and naked only reached as a fallback:

```
function getOptionType(bool _isPut, address collateral, address underlying)
↪ internal view returns (OptionType) {
    WhitelistInterface whitelist = WhitelistInterface(addressBook.getWhitelist());
    bool covered = whitelist.isCoveredWhitelistedCollateral(collateral, underlying,
↪ _isPut);
    bool naked = whitelist.isNakedWhitelistedCollateral(collateral, underlying,
↪ _isPut);
    if (_isPut && covered) {
        return OptionType.PUT;
    } else if (!_isPut && covered) {
        return OptionType.COVERED_CALL;
    } else if (_isPut && naked) {
        return OptionType.NAKED_PUT;
    } else if (!_isPut && naked) {
        return OptionType.NAKED_CALL;
    } else {
        revert("invalid option type");
    }
}
```

```

    }
}

```

The two whitelists are independent mappings, and neither setter clears or rejects the other for the same product hash:

```

function whitelistCoveredCollateral(address _collateral, address _underlying, bool
↳ _isPut) external onlyOwner {
    require(_isPut || _collateral == _underlying, "Whitelist: covered call
↳ collateral must be underlying");
    coveredWhitelistedCollateral[keccak256(abi.encode(_collateral, _underlying,
↳ _isPut))] = true;
    ...
}

function whitelistNakedCollateral(address _collateral, address _underlying, bool
↳ _isPut) external onlyOwner {
    nakedWhitelistedCollateral[keccak256(abi.encode(_collateral, _underlying,
↳ _isPut))] = true;
    ...
}

```

So a tuple whitelisted as both covered and naked has `covered == true`, and `getOptionType` returns `PUT / COVERED_CALL`, never the naked variant.

`getOptionType` only feeds the naked-margin path. In `_getMarginRequired`, the naked branch (`vaultType == 1 && !hasLong`) uses it to pick the formula in `_getNakedMarginRequired`, where the covered and naked branches compute materially different requirements (the `COVERED_CALL` branch caps the requirement at one collateral unit per option; `NAKED_CALL` derives it from `shortStrike * spotShock` against the underlying price; the put branches differ similarly). If a naked vault's collateral were ambiguously whitelisted, the covered formula would be applied and the vault would be margined too low. The only other caller, the liquidation function, assigns the result to a local that is never read, so in practice `getOptionType` influences behavior solely through the naked-margin computation.

Both the ambiguity and its consequence are confined to functionality that the supported configuration does not exercise: only the covered-call path is whitelisted, and type 1 naked-margin vaults cannot be opened through the live entry points (EnhancedOptions opens only type 0/2 vaults, and direct `Controller.operate` calls are restricted to the manager while `operatorsEnabled` is false). The dual-whitelist condition also requires a deliberate owner action that the supported setup does not take. With covered-call only and collateral constrained to equal the underlying, the naked formulas are never selected and the ambiguity cannot manifest.

Impact

None in the audited configuration. The behavior is a latent correctness/design weakness inherited from the core margin system: if naked-margin (type 1) support were enabled and the owner whitelisted the same product as both covered and naked, naked positions on that product would be under-margined (computed with the covered formula), appearing healthy and non-liquidatable when the naked model would demand more collateral, with any shortfall borne by the pool. Because the protocol configures only the covered-call path and does not create type 1 vaults, this cannot occur as deployed.

Code Snippet

<https://github.com/sherlock-audit/2026-05-enhanced-may-26th-2026/blob/b2e0082f695d5f0a82d8a5051d69423ad4a7cba4/contracts/src/core/MarginCalculator.sol#L131-L1326>

<https://github.com/sherlock-audit/2026-05-enhanced-may-26th-2026/blob/b2e0082f695d5f0a82d8a5051d69423ad4a7cba4/contracts/src/core/Whitelist.sol#L213-L231>

<https://github.com/sherlock-audit/2026-05-enhanced-may-26th-2026/blob/b2e0082f695d5f0a82d8a5051d69423ad4a7cba4/contracts/src/core/MarginCalculator.sol#L689-L700>

Tool Used

Manual Review

Recommendation

No change is required for the current covered-call-only deployment. If naked-margin (type 1) support is enabled later, make the option-type resolution explicit instead of relying on whitelist priority by passing the vault type into the resolution so a type 1 vault always uses the naked formula and a type 0/2 vault always uses the covered formula, regardless of whitelist membership.

Separately, the unused `getOptionType` call in the liquidation function can be removed, since its result is discarded.

Discussion

6xiaowu9

Currently, this issue does not exist.

`getOptionType` prioritizes covered when both covered and naked are true, which makes the local logic ambiguous. However, only the type-1 naked margin calculation would rely on this result in a way that could introduce risk.

At the moment, EnhancedOptions does not create type-1 options, and regular users cannot directly call the Controller to create them.

In addition, the double whitelist would still require an active misconfiguration by the owner. Under the current complete entry flow and configuration, there is no exploitable impact.

cwkang1998

We acknowledge this as low/informational, as there are no plans of supporting naked calls.

Issue L-23: Redundant shortAmount equals longAmount requirement in the strike-collateralized call branch [ACKNOWLEDGED]

Source:

<https://github.com/sherlock-audit/2026-05-enhanced-may-26th-2026/issues/74>

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

`_getCallSpreadMarginRequired` enforces `shortAmount == longAmount` in the branch taken when the call's collateral asset equals its strike asset. This requirement runs on every vault update for that branch and would block any unequal-leg (asymmetric) cash-settled call spread. The branch is not reachable through the supported entry points, so it does not actually prevent any supported vault, but the requirement is legacy code that does not belong on a general margin path.

Vulnerability Detail

In the call branch of `_getCallSpreadMarginRequired`, when `strikeAssetIsCollateral` is true:

```
if (strikeAssetIsCollateral) {
  // be certain the amounts are the same as the collateral requirements become
  ↳ problematic
  require(_shortAmount.isEqual(_longAmount), "MarginCalculator: long and short
  ↳ amounts must be the same");
  return FPI.max((_longStrike).sub(_shortStrike).mul(_shortAmount), ZERO);
}
```

This function is reached from `_getMarginRequired`, which is called by `getExcessCollateral` and therefore by `_verifyFinalState` on every vault-changing operation. So for a strike-collateralized call spread, any vault state with a long leg of a different size than the short leg would revert, even when the resulting position is a perfectly valid, more-conservative spread.

Per the protocol team, this is not a constraint on any currently supported vault:

- The `EnhancedOptions` entry point creates short-only vaults and does not expose long-oToken deposits.
- Type 2 vaults explicitly reject long-oToken deposits.
- For calls, `whitelistCoveredCollateral` only permits collateral equal to the underlying, so (assuming the normal case where underlying and strike differ) the `collateralAsset == strikeAsset` call branch is not reachable.

Underlying-collateralized type 0 call spreads take the other branch, which has no equality check.

So the requirement sits exclusively in an unreachable branch and does not block supported cash-settled call spreads.

Impact

No impact under the current entry-point and whitelist constraints. The check lives only in the strike-collateralized call branch, which cannot be reached for supported products, so it never rejects a valid vault. If a strike-collateralized call vault type were ever exposed through a future manager entry point, this requirement would forbid asymmetric call spreads on a code path that runs on every vault update, with no economic justification (an unequal spread is a valid, bounded-risk position).

Code Snippet

<https://github.com/sherlock-audit/2026-05-enhanced-may-26th-2026/blob/b2e0082f695d5f0a82d8a5051d69423ad4a7cba4/contracts/src/core/MarginCalculator.sol#L917-L921>

Tool Used

Manual Review

Recommendation

Treat this as cleanup. If the strike-collateralized call branch is genuinely unreachable, remove it together with the denomination handling discussed in the companion finding. If a strike-collateralized call vault type is ever intended, replace the hard equality requirement with the general unequal-leg call-spread formula, so asymmetric spreads are permitted and correctly margined.

Discussion

6xiaowu9

No security fix is required for the current supported product set.

cwkang1998

We acknowledge this as low/informational, as this is not part of the supported protocol flow.

Issue L-24: Call spread margin returned in underlying units is not converted for a third-asset collateral [ACKNOWLEDGED]

Source:

<https://github.com/sherlock-audit/2026-05-enhanced-may-26th-2026/issues/75>

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

For a call, `_getCallSpreadMarginRequired` returns the required margin denominated in underlying (oToken) units in its general branch. `_getMarginRequired` converts the result only when the collateral asset equals the strike asset, and that conversion is a mislabeled no-op. If a call's collateral asset were neither the underlying nor the strike asset, the underlying-denominated requirement would be returned as-is and treated as a collateral-asset amount, which is the wrong denomination. The covered-collateral whitelist prevents this configuration for any reachable call vault, so it does not affect collateral accounting today.

Vulnerability Detail

The general (non-strike-collateralized) branch of `_getCallSpreadMarginRequired` returns a value in underlying/oToken units, and its own comment states the caller is expected to convert it:

```
// D27{oToken} = max(D27{oToken}, D27{oToken}); caller converts to collateral via
↪ _convertAmountOnLivePrice
return FPI.max(firstPart, secondPart); // D27{oToken}
```

But `_getMarginRequired` only performs a conversion in the `collateralAsset == strikeAsset` case, and there it converts between the strike and collateral assets, which are equal in that branch, so the conversion is a no-op and its comment is inaccurate:

```
if (otokenDetails.otokenCollateralAsset == otokenDetails.otokenStrikeAsset) {
    // collateral needed is denominated in underlying asset so we need to convert
    ↪ to strike asset
    return (collateralAmount, _convertAmountOnLivePrice(assetNeeded,
        ↪ otokenDetails.otokenStrikeAsset, otokenDetails.otokenCollateralAsset));
}
return (collateralAmount, assetNeeded);
```

So:

- collateral == underlying: the underlying-denominated requirement is already in collateral units, and returning `assetNeeded` directly is correct.
- collateral == strike: the strike-collateralized branch returns a strike-denominated value and converts strike to collateral, equal assets, a no-op.
- collateral is a third asset (not underlying, not strike): `assetNeeded` is returned directly with no conversion from underlying units into the collateral asset, so the requirement is in the wrong denomination.

Per the protocol team, the third-asset case cannot be reached. For calls, `whitelistCoveredCollateral` requires `collateral == underlying`, and `_isMarginableCollateral` re-checks covered collateral during final-state validation, so a call vault whose collateral differs from both the strike and the underlying is rejected before it can pass validation. The team also confirms the strike-collateralized branch is unreachable for supported products and that the no-op conversion and its comment are inaccurate legacy code.

Impact

No impact under the current whitelist and entry-point constraints. Every reachable call vault uses collateral equal to the underlying, where the underlying-denominated result is already correct, so no conversion is required and none is missing. The issue is latent: if a call vault were ever allowed to use a collateral asset distinct from both the strike and the underlying, the margin requirement would be computed in underlying units and consumed as collateral units, mispricing the collateral requirement. The inaccurate comment and no-op conversion in the strike-collateralized branch also obscure the actual denomination on this path.

Code Snippet

<https://github.com/sherlock-audit/2026-05-enhanced-may-26th-2026/blob/b2e0082f695d5f0a82d8a5051d69423ad4a7cba4/contracts/src/core/MarginCalculator.sol#L717-L735>

<https://github.com/sherlock-audit/2026-05-enhanced-may-26th-2026/blob/b2e0082f695d5f0a82d8a5051d69423ad4a7cba4/contracts/src/core/MarginCalculator.sol#L938-L939>

Tool Used

Manual Review

Recommendation

Treat as cleanup under the current constraints. Either remove the unreachable strike-collateralized call branch and its no-op conversion, or, if call collateral may ever

differ from the underlying, make the denomination explicit: keep `_getCallSpreadMarginRequired` returning underlying units and have `_getMarginRequired` convert from the underlying asset to the actual collateral asset in every case where they differ, rather than only when collateral equals strike. Fix the comment at the conversion site so it describes the real source and target denominations.

Discussion

6xiaowu9

No immediate fix is required for the current supported product set.

cwkang1998

We acknowledge this as low/informational, as this is unreachable for our supported protocol flow.

Issue L-25: Unused local variables in the liquidation check function [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-05-enhanced-may-26th-2026/issues/76>

Summary

The function that computes a vault's liquidation state declares two local variables, `productHash` and `opType`, that are assigned but never read. They add unnecessary computation (including an extra `getOptionType` call and a `keccak256` hash) and reduce readability.

Vulnerability Detail

After building `shortDetails`, the function computes `productHash`:

```
bytes32 productHash = _getProductHash(  
    vaultDetails.shortUnderlyingAsset,  
    vaultDetails.shortStrikeAsset,  
    vaultDetails.shortCollateralAsset,  
    vaultDetails.isShortPut  
);
```

`productHash` is never used afterwards. The product hash needed for the margin computation is recalculated independently inside `_getMarginRequired()`, so this assignment is dead.

A few lines later, `opType` is computed and also never used:

```
OptionType opType = getOptionType(  
    vaultDetails.isShortPut, vaultDetails.shortCollateralAsset,  
    ↪ vaultDetails.shortUnderlyingAsset  
);
```

Neither variable feeds into the liquidation decision (`collateralRequired` vs `depositedCollateral`) or the returned `debtPrice`.

Impact

No security impact. This is a code-quality and gas issue: each call to this function performs a redundant `keccak256` hash and a redundant `getOptionType` lookup whose results are discarded.

Code Snippet

<https://github.com/sherlock-audit/2026-05-enhanced-may-26th-2026/blob/b2e0082f695d5f0a82d8a5051d69423ad4a7cba4/contracts/src/core/MarginCalculator.sol#L503-L508>

<https://github.com/sherlock-audit/2026-05-enhanced-may-26th-2026/blob/b2e0082f695d5f0a82d8a5051d69423ad4a7cba4/contracts/src/core/MarginCalculator.sol#L515-L517>

Tool Used

Manual Review

Recommendation

Remove both unused assignments:

```
bytes32 productHash = _getProductHash(  
    vaultDetails.shortUnderlyingAsset,  
    vaultDetails.shortStrikeAsset,  
    vaultDetails.shortCollateralAsset,  
    vaultDetails.isShortPut  
);
```

```
OptionType opType = getOptionType(  
    vaultDetails.isShortPut, vaultDetails.shortCollateralAsset,  
    ↪ vaultDetails.shortUnderlyingAsset  
);
```

Discussion

6xiaowu9

Fixed. Commit 722cb49 (fix: remove the unused local variable)

Issue L-26: Redundant no-op call to removeExcessCollateralFromRedemptionBalances on ITM put redeem [RESOLVED]

Source:

<https://github.com/sherlock-audit/2026-05-enhanced-may-26th-2026/issues/77>

Summary

In the put branch of `handleRedeem`, `_removeExcessCollateralFromRedemptionBalances` is called, but for an ITM put that function always returns early without changing any state. The call is a no-op and can be removed.

Vulnerability Detail

The put redeem branch first requires the put to be ITM, then calls the excess-removal helper:

```
if (otoken.isPut()) {
    // check put is ITM
    require(underlyingExpiryPrice < strikePrice, "C45");
    // take the underlying asset payment
    _removeExcessCollateralFromRedemptionBalances(
        otoken, strikePrice, underlyingExpiryPrice, collateralExpiryPrice
    );
    pool.transferToPool(otoken.underlyingAsset(), _args.receiver, strikePayment);
}
```

Inside the helper, the OTM branch requires `otoken.isPut() && underlyingExpiryPrice >= strikePrice`, which can never hold here because the redeem branch already enforced `underlyingExpiryPrice < strikePrice`. Execution therefore always reaches the put case, which returns immediately:

```
} else if (otoken.isPut()) {
    // dont need to handle ITM put cases because collateralAsset == strikeAsset, so
    //   ↳ fully collateralised with no excess.
    return;
}
```

So the call performs only a `pool.getRedemptionBalance` read and a couple of branch comparisons, then returns without updating anything.

Impact

No security impact. This is a minor gas and readability issue: each ITM put redeem makes a call that cannot change state.

Code Snippet

<https://github.com/sherlock-audit/2026-05-enhanced-may-26th-2026/blob/b2e0082f695d5f0a82d8a5051d69423ad4a7cba4/contracts/src/core/ControllerLogic.sol#L364-L366>

<https://github.com/sherlock-audit/2026-05-enhanced-may-26th-2026/blob/b2e0082f695d5f0a82d8a5051d69423ad4a7cba4/contracts/src/core/ControllerLogic.sol#L578-L581>

Tool Used

Manual Review

Recommendation

Remove the call from the ITM put branch:

```
if (otoken.isPut()) {
    // check put is ITM
    require(underlyingExpiryPrice < strikePrice, "C45");
    _removeExcessCollateralFromRedemptionBalances(
        otoken, strikePrice, underlyingExpiryPrice, collateralExpiryPrice
    );
    // take the underlying asset payment
    pool.transferToPool(otoken.underlyingAsset(), _args.receiver, strikePayment);
}
```

Keep the call only in the call-option branch, where the helper performs real work.

Discussion

6xiaowu9

This is not an issue.

The calculation discrepancy exists at a purely arithmetic level, but the real product configuration required by the comment cannot be established through the complete entry flow.

Physically settled positions are opened by EnhancedOptions as vault type 2, and must ultimately pass through Controller._verifyFinalState() and

`MarginCalculator._isMarginableCollateral()`. For calls, `Whitelist.whitelistCoveredCollateral(collateral, underlying, false)` explicitly requires `collateral == underlying`; otherwise, it reverts with `Whitelist: covered call collateral must be underlying`. Therefore, the scenario described in the comment – “ETH as the underlying and an appreciating 6-decimal LST as the collateral” – cannot become a valid type 2 vault.

If the underlying and collateral are the same 6-decimal asset, their expiry prices are also the same, and the appreciation-based excess calculated by `_removeExcessCollateralFromRedemptionBalances()` is zero. Even if downward rounding from multiple mints causes the equality check to return early, there is no appreciation surplus that should be removed from the redemption balance, so the alleged stuck-funds issue does not occur.

Disclaimers

Sherlock does not provide guarantees nor warranties relating to the security of the project.

Usage of all smart contract software is at the respective users' sole risk and is the users' responsibility.